

HLD (High-Level Design) in Go - పూర్తి తెలుగు గైడ్ (SDE2 & SSE)

ఈ document చదివిన తర్వాత Go లో **production backend systems** ఎలా **design & scale** చేయాలో మళ్ళీ మర్చిపోలేవు. ప్రతి concept కీ real-life analogy, ఎప్పుడు/ఎందుకు వాడాలి, idiomatic runnable Go code, trade-offs, ASCII diagram, మరియు interview దృష్టి ఉంటాయి.

Focus: production Go services (net/http, gRPC, database/sql, worker pools, graceful shutdown) + distributed-systems building blocks (load balancing, caching, replication, sharding, CAP, consensus) - అన్నీ **Go lens** తో. అంటే "concept ఏమిటి" మాత్రమే కాదు, "Go లో దీన్ని ఎలా **implement/use** చేస్తాం" అని.

సోదర documents:

- [GO_Telugu.md](#) - Go language basics (goroutines, channels, interfaces). ఇది ముందు చదువు - ముఖ్యంగా concurrency chapter. HLD-in-Go అర్థం కావాలంటే goroutines/channels/context పునాది కావాలి.
- [LLD Go Telugu.md](#) - Low-Level Design in Go (interfaces, patterns, clean architecture).
- [SystemDesign Go Telugu.md](#) - full case studies (URL shortener, chat, feed) Go implementations తో.

లక్ష్యం: SDE2 (mid-level) మరియు SSE (Senior Software Engineer) rounds ని confident గా clear చేయడం - "Go ఎందుకు cloud-native king" అనేది నీ ఎముకల్లో నాటుకునేలా.

విషయ సూచిక (Table of Contents)

Part 1 — Go for Backend (ఎందుకు Go)

1. HLD in Go context — ఎందుకు Go cloud-native king
2. Anatomy of a production Go service — layout, config, 12-factor
3. Graceful startup & shutdown — signals, context, draining, health/readiness

Part 2 — Building the Service Layer

4. net/http deep — server, ServeMux, handlers, middleware, timeouts
5. Routing & middleware frameworks (chi/gin/echo) + custom middleware
6. gRPC in Go — protobuf, unary & streaming, interceptors
7. API design in Go — REST vs gRPC vs GraphQL, versioning, pagination, idempotency
8. Realtime in Go — WebSockets, SSE, long polling

Part 3 — Data & Caching Layer

9. database/sql deep — pool tuning, prepared statements, context
10. ORMs & query builders (GORM, sqlx, sqlc)
11. Caching in Go — in-process, Redis, singleflight
12. Message queues in Go — Kafka, NATS, RabbitMQ

Part 4 — Distributed Systems Building Blocks (Go lens)

13. Load balancing & service discovery (L4/L7, client-side LB, Consul/etcd)
14. SQL vs NoSQL, replication, sharding/partitioning
15. Consistency, CAP/PACELC, consensus (Raft) — etcd/hashicorp-raft
16. Idempotency, distributed locks, distributed transactions (Saga)

Part 5 — Resilience & Concurrency at Scale

- 17. Concurrency at scale — worker pools, bounded concurrency, errgroup, fan-out/fan-in
- 18. Resilience patterns — timeouts, retries, circuit breaker, bulkhead, rate limiting
- 19. Load shedding & graceful degradation

Part 6 — Observability & Operations

- 20. Structured logging (slog, zap, zerolog), correlation IDs
- 21. Metrics (Prometheus client_golang, RED/USE), Grafana
- 22. Distributed tracing (OpenTelemetry), context propagation
- 23. Profiling in production (pprof), runtime metrics

Part 7 — Deploy & Scale

- 24. Containerizing Go — multi-stage, distroless/scratch, CGO_ENABLED
- 25. Running Go in Kubernetes — GOMAXPROCS, GOMEMLIMIT, probes
- 26. Scaling Go services — statelessness, autoscaling, GC/allocation tuning
- 27. Security basics — TLS, secrets, validation, JWT/OAuth

Part 8 — Reference

- 28. HLD-in-Go interview framework + checklist + Memory Tips + Common Mistakes + numbers cheat sheet

Part 1 — Go for Backend (ఎందుకు Go)

1. HLD in Go context — ఎందుకు Go cloud-native king

వివరణ

HLD అంటే మొత్తం system ని పెద్ద బొమ్మ స్థాయిలో design చేయడం - ఏ services, ఏ databases, ఏ caches, అవి ఎలా మాట్లాడుకుంటాయి, ఎలా scale అవుతాయి. కానీ ఈ guide లో ప్రతి decision ని **Go అనే lens** లోంచి చూస్తాం - ఎందుకంటే modern cloud-native infrastructure దాదాపు అంతా Go లో రాయబడింది.

Go ఎందుకు backend/cloud-native world ని ఏలుతోంది - నాలుగు స్తంభాలు:

- 1. Fast compile + single static binary** - Go program `go build` చేస్తే ఒక్క self-contained binary వస్తుంది. JVM లేదు, `node_modules` లేదు, interpreter లేదు. `scratch / distroless` image లో ఆ ఒక్క binary పెడితే container 10-20 MB. Deploy = ఒక్క file copy.
- 2. Concurrency built-in** - goroutines (KB-level stack, millions అవుతాయి) + channels + `go` scheduler. "Thread per request" costly అయిన Java/threads తో పోలిస్తే, Go లో ఒక్క machine మీద లక్షల concurrent connections handle చేయొచ్చు.
- 3. Low memory + fast startup** - GC ఉంది కానీ tuned (low pause). Startup milliseconds లో. అందుకే serverless/autoscaling కి perfect - cold start చిన్నది.
- 4. Static typing + simplicity** - ఒక్క way to do things, `gofmt`, small language. Large teams లో codebase consistent గా ఉంటుంది - "boring is good" for infra.

ఇవి Go లోనే రాయబడ్డాయి (ఇదే biggest proof): **Docker, Kubernetes, etcd, Prometheus, Grafana Loki, Terraform, Consul, Vault, CockroachDB, InfluxDB, containerd, CoreDNS, Istio, Helm, cri-o, Traefik, NATS, TiDB**. అంటే modern infrastructure నడిచే platform అంతా Go. ఈ tools తో integrate అవ్వాలంటే (client libraries, operators, exporters) - Go మొదటి-class citizen.

Real-life Scenario

Go binary = రెడీమేడ్ tiffin box. Java/JVM అంటే "వంటగది మొత్తం (JVM runtime) తో పాటు వంట తీసుకెళ్ళడం" - ఎక్కడ తినాలన్నా ముందు వంటగది setup చేయాలి. Node అంటే "సగం-వండిన సామాను + అన్ని ingredients (node_modules) మోసుకెళ్ళడం". Go binary అంటే "వండిన భోజనం ఒక్క box లో సీల్ చేసి ఇచ్చినట్టు" - ఎక్కడైనా box తెరిచి తినొచ్చు, extra ఏదీ అవసరం లేదు. అందుకే container లో Go image అంత చిన్నది, అంత fast.

Go vs Java vs Node — backend పోలిక

అంశం	Go	Java (Spring)	Node.js
Concurrency model	goroutines + channels (M:N scheduler)	threads (heavy) / virtual threads (కొత్త)	single-thread event loop + async
CPU-bound work	బాగా parallel (అన్ని cores)	parallel (threads)	weak (ఒక్క thread; worker threads కష్టం)
Memory footprint	తక్కువ (10s of MB)	ఎక్కువ (JVM heap, 100s MB)	మధ్యస్థం
Startup time	milliseconds	seconds (JVM warmup)	fast
Deploy artifact	ఒక్క static binary	JAR + JVM	JS + node_modules + runtime
Container image	10-20 MB (scratch)	200 MB+	100 MB+
Type safety	static, compiled	static, compiled	dynamic (TS optional)
Error handling	explicit if err != nil	exceptions	exceptions / promises
Ecosystem (cloud infra)	native (k8s, docker...)	mature enterprise	web/JS-heavy
Learning curve	చిన్నది (small language)	పెద్దది (Spring magic)	చిన్నది (కానీ async traps)
Best fit	microservices, infra, high-concurrency APIs, CLIs	enterprise, complex domains	I/O-bound web, realtime, BFF

గుర్తుంచుకో: Go "silver bullet" కాదు. Heavy computation + rich domain modeling + huge existing ecosystem కావాలంటే Java గొప్పది. చాలా small teams కి Node వేగంగా ship చేస్తుంది. కానీ **network-heavy, high-concurrency, cloud-native, ops-friendly** services కి Go sweet spot - low latency, low memory, easy deploy.

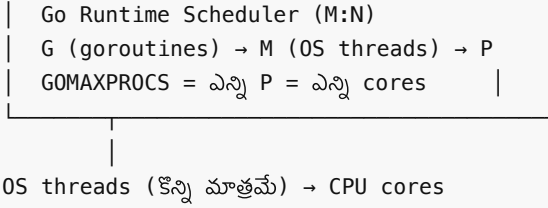
Go program ఎలా నడుస్తుంది (mental model)

```

main() goroutine
├── go handler1()
├── go handler2()
├── go handler3()
└── ...

```

}
 లక్షల goroutines
 (ఒక్కటే ~2KB stack, పెరుగుతుంది)



- **G** = goroutine, **M** = OS thread (machine), **P** = processor (logical, GOMAXPROCS count).
- Goroutine block అయితే (network read) scheduler దాన్ని park చేసి M ని వేరే G కి ఇస్తుంది - అందుకే లక్షల connections ఒక్క machine లో.

Trade-offs

- **✓ లాభాలు:** deploy simple, memory తక్కువ, concurrency native, cloud ecosystem, fast build/CI.
- **✗ నష్టాలు:** generics ఇటీవలే (1.18), error handling verbose (`if err != nil` పడేపడే), GC ఉంది కాబట్టి hard-realtime కాదు, dependency management పాత versions లో గందరగోళం (ఇప్పుడు modules తో fixed), rich frameworks తక్కువ (Go "library over framework" culture).

Key Points

- Go dominates cloud-native ఎందుకంటే: **static binary + concurrency + low memory + fast start**.
- Docker/K8s/etcd/Prometheus అన్నీ Go - infra తో integrate అవ్వాలంటే Go native.
- HLD-in-Go = classic building blocks (LB, cache, shard...) + **Go-specific concerns** (GOMAXPROCS, goroutine leaks, context propagation, pool tuning).
- Go "boring" గా ఉండటమే దాని బలం - large teams లో consistency.

Interview దృష్టి

"ఈ service ఎందుకు Go లో?" అని అడిగితే: "High-concurrency network I/O + తక్కువ memory footprint + fast autoscaling (small cold start) + మా infra Kubernetes/Prometheus ecosystem తో native integration. Per-request thread cost లేకుండా goroutines తో లక్షల concurrent connections handle చేయగలం." అని answer చేయి. Language ని blindly praise చేయకు - **workload fit** చెప్పు.

2. Anatomy of a production Go service — layout, config, 12-factor

వివరణ

Toy Go program `main.go` ఒక్క file. కానీ production service కి **structure** కావాలి - testable, maintainable, team-friendly. Go community లో ఒక de-facto layout ఉంది (గుడ్డిగా follow అవసరం లేదు, కానీ conventions తెలుసుకో).

ముఖ్య directories:

- **cmd/** - entry points. ఒక్కో binary కి ఒక subfolder: `cmd/api/main.go` , `cmd/worker/main.go` . main package చిన్నదిగా ఉండాలి - wiring మాత్రమే.
- **internal/** - నీ private code. Go **compiler enforce** చేస్తుంది: `internal/` లోని packages ని ఆ module బయటి ఎవరూ import చేయలేరు. Business logic ఇక్కడ.
- **pkg/** - బయటివాళ్ళు కూడా వాడగల reusable libraries (public API). చాలామంది దీన్ని అవసరం లేకపోతే వాడొద్దంటారు.
- **api/** - protobuf/OpenAPI definitions.
- **configs/** , **deployments/** , **migrations/** - config templates, k8s manifests, SQL migrations.

Real-life Scenario

internal/ = ఇంటి లోపలి గదులు, **cmd/** = ఇంటి తలుపులు. ఇంటికి చాలా తలుపులు (front door = API server, back door = background worker) ఉండొచ్చు - కానీ లోపలి గదులు (business logic) ఒకటి, అన్ని తలుపుల నుంచి access అవుతాయి. **internal/** అనే గోడ ఉంది - పొరుగువాళ్ళు (వేరే modules) నీ bedroom (private packages) లోకి రాలేరు, compiler security guard లా అడ్డుకుంటుంది.

Production layout (ఉదాహరణ)

```
myservice/
├─ cmd/
│   ├── api/main.go      # HTTP/gRPC server endpoint (చిన్నది - wiring)
│   └─ worker/main.go    # background consumer endpoint
├─ internal/             # private (బయటి modules import చేయలేవు)
│   ├── config/config.go # env → struct
│   ├── httpapi/         # handlers, middleware, router
│   ├── service/         # business logic (pure, testable)
│   ├── store/           # DB access (repository interfaces)
│   └─ platform/         # logger, metrics, tracing setup
├─ pkg/                  # public reusable (ఐచ్ఛికం)
├─ api/proto/            # .proto files
├─ migrations/           # 0001_init.up.sql ...
├─ deployments/k8s/      # manifests
├─ go.mod
├─ go.sum
├─ Dockerfile
└─ Makefile
```

Config — 12-factor (env-first)

12-factor app rule: **config** ని **environment** లో పెట్టు, **code** లో కాదు. అదే code, వేరే env vars → dev/staging/prod. Go లో సాధారణంగా env vars + flags, పెద్దవాటికి **viper** .

```
// internal/config/config.go
package config

import (
    "fmt"
    "time"

    "github.com/kelseyhightower/envconfig" // env → struct (popular, చిన్నది)
)

// Config = అన్ని runtime settings ఒక్క struct లో (typed, validated)
type Config struct {
    Env                string        `envconfig:"ENV" default:"dev"`
    HTTPAddr           string        `envconfig:"HTTP_ADDR" default:":8080"`
    ReadTimeout        time.Duration `envconfig:"READ_TIMEOUT" default:"5s"`
    WriteTimeout       time.Duration `envconfig:"WRITE_TIMEOUT" default:"10s"`
    ShutdownTimeout    time.Duration `envconfig:"SHUTDOWN_TIMEOUT" default:"15s"`

    DatabaseURL        string        `envconfig:"DATABASE_URL" required:"true" // secret -
    env నుంచి
```

```

DBMaxOpenConns int    `envconfig:"DB_MAX_OPEN_CONNS" default:"25"`
RedisAddr      string `envconfig:"REDIS_ADDR" default:"localhost:6379"`
LogLevel       string `envconfig:"LOG_LEVEL" default:"info"`
}

// Load = env చదివి, validate చేసి, typed struct ఇస్తుంది
func Load() (Config, error) {
    var c Config
    if err := envconfig.Process("", &c); err != nil { // MYAPP_ prefix కూడా పెట్టొచ్చు
        return Config{}, fmt.Errorf("config load: %w", err)
    }
    // అదనపు validation (fail-fast - startup లోనే తప్పు పట్టుకో)
    if c.DBMaxOpenConns <= 0 {
        return Config{}, fmt.Errorf("DB_MAX_OPEN_CONNS must be > 0")
    }
    return c, nil
}

```

```

// cmd/api/main.go - చిన్నది, wiring మాత్రమే
package main

import (
    "log"

    "myservice/internal/config"
)

func main() {
    cfg, err := config.Load()
    if err != nil {
        log.Fatalf("config: %v", err) // startup లోనే fail (12-factor: crash loudly)
    }
    // ...logger, db, server setup (తర్వాతి topics లో)
    _ = cfg
}

```

config అందించే మార్గాలు — పోలిక

మార్గం	ఎప్పుడు	Go tool
Env vars	12-factor default, secrets, k8s ConfigMap/Secret	os.Getenv, envconfig
Flags	CLI tools, local overrides	stdlib flag, pflag
Config file (YAML/TOML)	పెద్ద nested config, local dev	viper, koanf
Secret manager	passwords, API keys (prod)	Vault SDK, cloud SDK

Precedence order (సాధారణంగా): flags > env > config file > defaults. `viper` దీన్ని native గా చేస్తుంది.

12-factor — Go కి ముఖ్యమైనవి

Factor	Go లో అర్థం
Config in env	secrets/URLs env నుంచి, hardcode కాదు
Stateless processes	in-memory session లేదు; state DB/Redis లో → horizontal scale
Port binding	service తనే HTTP port bind చేస్తుంది (http.Server), external web server అవసరం లేదు
Logs as streams	stdout/stderr కి log (slog); file rotation infra చూసుకుంటుంది
Disposability	fast startup + graceful shutdown (Topic 3)
Dev/prod parity	same binary, config మాత్రమే మారుతుంది

Trade-offs

- **internal/ వాడాలా?** ✅ business logic ని accidental import నుంచి కాపాడుతుంది. చిన్న single-binary tool కి over-kill కావొచ్చు.
- **pkg/ వాడాలా?** బయటివాళ్ళు వాడే library అయితేనే. లేకపోతే **internal/** చాలు (చాలామంది **pkg/** skip చేస్తారు).
- **Flat vs deep layout:** చిన్న service ని over-structure చేయకు. అవసరమైనప్పుడు split చేయి (YAGNI).

Key Points

- **cmd/** = entrypoints (చిన్నవి), **internal/** = private logic (compiler-enforced), **pkg/** = public (ఐచ్ఛికం).
- **Config env నుంచి**, typed struct లోకి load చేసి, **startup లో validate** (fail-fast).
- Secrets ని code/git లో పెట్టకు - env/secret-manager నుంచి.
- **main** package thin గా ఉంచు - dependencies wire చేసి inject చేయి (testability).

Interview దృష్టి

"Config ఎలా manage చేస్తావ్?" → "12-factor: env vars, typed struct లోకి load, startup లో validate (fail-fast). Secrets Vault/k8s Secret నుంచి, code లో ఎప్పుడూ కాదు. Same binary అన్ని envs లో - config మాత్రమే మారుతుంది." Dependency injection చెప్పు (constructor లోకి pass) - global state కాదు, testable.

3. Graceful startup & shutdown — signals, context, draining, health/readiness

వివరణ

Kubernetes ఒక pod ని ఎప్పుడైనా చంపొచ్చు (deploy, scale-down, node drain). అది **SIGTERM** పంపుతుంది, తర్వాత grace period (default 30s), తర్వాత **SIGKILL** (force). నీ service SIGTERM విని **గ్రేస్‌ఫుల్‌గా** ఆగాలి:

1. కొత్త requests తీసుకోవడం ఆపు (readiness fail → LB traffic పంపడం మానేస్తుంది).
2. **in-flight requests drain** అవ్వనివ్వ (అవి పూర్తయ్యేదాకా wait).
3. DB pools, queue consumers, connections **close** చేయి.
4. timeout లోపు clean exit; timeout దాటితే force.

లేకపోతే: mid-request users కి 500లు, half-written DB rows, leaked connections.

Health vs Readiness (Kubernetes probes) - రెండూ వేరు:

- **Liveness** - "process సజీవంగా ఉందా? Deadlock లో ఇరుక్కుందా?" Fail → k8s pod **restart** చేస్తుంది.
- **Readiness** - "ఇప్పుడు traffic తీసుకోగలనా? (DB connected? warmed up?)" Fail → k8s **traffic ఆపుతుంది** (restart కాదు).

Real-life Scenario

Shop మూసేయడం. యజమాని "ఇక మూసేస్తున్నా" (SIGTERM) అని చెప్పగానే - కొత్త customers ని లోపలికి రానివ్వడు (readiness = closed), కానీ **లోపల already ఉన్న customers ని బిల్లింగ్ పూర్తయ్యేదాకా ఉండనిస్తాడు** (drain in-flight). అందరూ వెళ్ళాక lights ఆపి తలుపు వేస్తాడు (close resources). ఎవరో బిల్లింగ్ లో చాలా సేపు తీసుకుంటే - deadline దాటాక "sorry, force close" (shutdown timeout → SIGKILL). Customer mid-transaction లో తలుపు వేయడం = data corruption.

Code — signal-aware graceful shutdown

```
// cmd/api/main.go
package main

import (
    "context"
    "errors"
    "log/slog"
    "net/http"
    "os"
    "os/signal"
    "sync/atomic"
    "syscall"
    "time"
)

func main() {
    logger := slog.New(slog.NewJSONHandler(os.Stdout, nil))

    // signal వస్తే cancel అయ్యే root context (Go 1.16+ idiomatic)
    ctx, stop := signal.NotifyContext(context.Background(),
        syscall.SIGINT, syscall.SIGTERM)
    defer stop()

    // readiness flag (atomic - concurrent safe). 0 = not ready, 1 = ready
    var ready atomic.Bool

    mux := http.NewServeMux()

    // liveness: process సజీవంగా ఉందా? (ఎప్పుడూ 200 - deadlock అయితేనే fail)
    mux.HandleFunc("/healthz", func(w http.ResponseWriter, r *http.Request) {
        w.WriteHeader(http.StatusOK)
    })

    // readiness: traffic తీసుకోగలనా? shutdown మొదలైతే fail → LB traffic ఆపుతుంది
    mux.HandleFunc("/readyz", func(w http.ResponseWriter, r *http.Request) {
        if !ready.Load() {
```

```

        http.Error(w, "not ready", http.StatusServiceUnavailable) // 503
        return
    }
    w.WriteHeader(http.StatusOK)
})

mux.HandleFunc("/work", func(w http.ResponseWriter, r *http.Request) {
    time.Sleep(2 * time.Second) // in-flight request simulate చేస్తోంది
    w.Write([]byte("done"))
})

srv := &http.Server{
    Addr:      ":8080",
    Handler:    mux,
    ReadTimeout: 5 * time.Second,
    WriteTimeout: 15 * time.Second,
    IdleTimeout: 60 * time.Second,
}

// server ని వేరే goroutine లో start (main ni block చేయకుండా)
go func() {
    ready.Store(true) // startup పనులు అయ్యాక (DB warm etc.) ready
    logger.Info("server starting", "addr", srv.Addr)
    if err := srv.ListenAndServe(); err != nil &&
        !errors.Is(err, http.ErrServerClosed) {
        logger.Error("listen failed", "err", err)
        stop() // fatal - shutdown trigger
    }
}()

// SIGTERM/SIGINT వచ్చేదాకా ఇక్కడ block
<-ctx.Done()
logger.Info("shutdown signal received")

// STEP 1: readiness fail చేయి → LB కొత్త traffic పంపడం ఆపుతుంది
ready.Store(false)
// LB probe interval కి కొంచెం సమయం ఇవ్వు (in-flight ఇంకా వస్తుంటాయి)
time.Sleep(2 * time.Second)

// STEP 2: bounded time లో in-flight requests drain
shutdownCtx, cancel := context.WithTimeout(context.Background(), 15*time.Second)
defer cancel()

// Shutdown = కొత్త connections ఆపి, active requests పూర్తయ్యేదాకా wait
if err := srv.Shutdown(shutdownCtx); err != nil {
    logger.Error("graceful shutdown timed out, forcing", "err", err)
    srv.Close() // force close (deadline దాటింది)
}

// STEP 3: మిగతా resources close (DB pool, kafka consumer, redis...)
// db.Close(); consumer.Close() ...

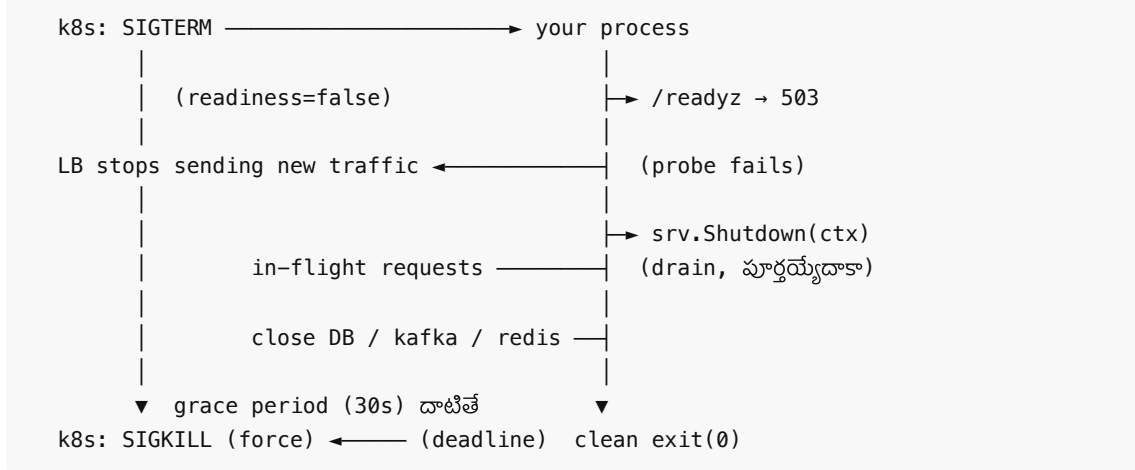
```

```

    logger.Info("shutdown complete")
}

```

shutdown sequence (diagram)



Liveness vs Readiness — పోలిక

అంశం	Liveness (/healthz)	Readiness (/readyz)
ప్రశ్న	process సజీవమా?	traffic తీసుకోగలనా?
Fail అయితే k8s	pod restart	traffic ఆపుతుంది (restart కాదు)
Check ఏమిటి	తేలికైనది (deadlock detect)	dependencies (DB, cache), shutdown flag
Shutdown time	200 (సజీవమే)	503 (traffic ఆగాలి)
Deep checks	❌ (dependency down → అనవసర restarts)	✅ (కానీ cascade జాగ్రత్త)

జాగ్రత్త: liveness లో DB check పెట్టకు! DB temporary down అయితే అన్ని pods restart అవుతూ cascade అవుతుంది. Liveness = "నా process హోంగ్ కాలేదు" మాత్రమే. Readiness లో dependency checks - అవి fail అయితే traffic ఆగుతుంది, restart కాదు.

Trade-offs

- Drain time ఎంత?** ఎక్కువ = safe కానీ deploy నెమ్మది. తక్కువ = fast కానీ requests కట్ అవుతాయి. `terminationGracePeriodSeconds` ని shutdown timeout కంటే ఎక్కువ పెట్టు.
- readiness=false తర్వాత sleep ఎందుకు?** LB probe interval (ఉదా 2-5s) ఉంది - వెంటనే Shutdown చేస్తే LB ఇంకా traffic పంపుతూ ఉంటుంది → race. అందుకే readiness fail చేసి కొద్దిసేపు ఆగి, తర్వాత drain.
- Long-running requests** (streaming, big uploads) shutdown timeout దాటొచ్చు - వాటికి context deadline propagate చేయి.

Key Points

- `signal.NotifyContext` తో SIGTERM/SIGINT ని root context cancel కి link చేయి.

- Shutdown order: **readiness off** → **wait** → **srv.Shutdown(ctx)** (**drain**) → **close resources**.
- Liveness ≠ readiness. Liveness fail = restart; readiness fail = traffic ఆపు. Liveness లో deep dependency checks పెట్టకు.
- `http.ErrServerClosed` normal shutdown - దాన్ని error గా log చేయకు.
- `terminationGracePeriodSeconds` > app shutdown timeout ఉండాలి.

Interview దృష్టి

"Zero-downtime deploy ఎలా?" → "Rolling update + readiness probe + graceful shutdown. SIGTERM వస్తే readiness ని fail చేసి LB traffic ఆపి, in-flight requests ని `srv.Shutdown(ctx)` తో drain చేసి, resources close చేస్తా. New pods readyz pass అయ్యాకే traffic. Liveness లో DB check పెట్టను - cascade restart వస్తుంది." ఇది senior signal - lifecycle + failure modes ఆలోచించావ్ అని చూపిస్తుంది.

Part 2 — Building the Service Layer

4. net/http deep — server, ServeMux, handlers, middleware, timeouts

వివరణ

Go లో web server కి **framework అవసరం లేదు** - `stdlib net/http` production-grade. Node లో Express, Java లో Spring లాంటి భారీ frameworks లేకుండా Go standard library తోనే scalable HTTP server నడుస్తుంది. ముఖ్య భాగాలు:

- **http.Handler** - ఒక్క interface: `ServeHTTP(w ResponseWriter, r *Request)`. అంతా దీని చుట్టూనే.
- **http.HandlerFunc** - ఒక function ని Handler గా మార్చే adapter.
- **ServeMux** - router (Go 1.22 నుంచి method + path patterns support: `"GET /users/{id}"`).
- **http.Server** - actual server, **timeouts** తో **configure చేయాలి** (defaults dangerous).
- **Middleware** - Handler ని wrap చేసే Handler (chain).

కీలకం: `http.ListenAndServe(":8080", mux)` వాడొద్దు production లో - అది `http.Server{}` default timeouts (zero = infinite) వాడుతుంది → slowloris attack, leaked connections. ఎప్పుడూ `http.Server{}` explicit timeouts తో వాడు.

Real-life Scenario

Middleware = airport security layers. Passenger (request) gate (handler) చేరేముందు - check-in (logging), security scan (auth), baggage tag (request-id), boarding pass verify (rate limit) - ఒక్కో layer దాటాలి. ప్రతి layer తర్వాత దాన్ని పీలుస్తుంది (`next.ServeHTTP`). ఏ layer అయినా "no entry" అంటే అక్కడే ఆగిపోతుంది (auth fail → 401, gate చేరదు). Response తిరిగి వచ్చేటప్పుడు కూడా అవే layers గుండా (response time log).

Code — production HTTP server + middleware chain

```
package main

import (
    "context"
    "log/slog"
    "net/http"
```

```

    "os"
    "time"

    "github.com/google/uuid"
)

// ---- Middleware = Handler తీసుకుని Handler ఇచ్చే function ----
type Middleware func(http.Handler) http.Handler

// Chain: middlewares ని ఒక handler చుట్టూ చుడుతుంది (బయటిది ముందు execute)
func Chain(h http.Handler, mws ...Middleware) http.Handler {
    for i := len(mws) - 1; i >= 0; i-- { // వెనక నుంచి wrap → మొదటిది బయట
        h = mws[i](h)
    }
    return h
}

// context key type (collision రాకుండా private type)
type ctxKey string

const requestIDKey ctxKey = "request_id"

// RequestID: ప్రతి request కి unique id (logs/traces correlate చేయడానికి)
func RequestID(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        id := r.Header.Get("X-Request-ID")
        if id == "" {
            id = uuid.NewString()
        }
        ctx := context.WithValue(r.Context(), requestIDKey, id)
        w.Header().Set("X-Request-ID", id)
        next.ServeHTTP(w, r.WithContext(ctx)) // కొత్త context తో ముందుకు
    })
}

// statusRecorder: status code capture చేయడానికి ResponseWriter wrap
type statusRecorder struct {
    http.ResponseWriter
    status int
}

func (s *statusRecorder) WriteHeader(code int) {
    s.status = code
    s.ResponseWriter.WriteHeader(code)
}

// Logging: ప్రతి request ని method, path, status, latency తో log
func Logging(logger *slog.Logger) Middleware {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            start := time.Now()
            rec := &statusRecorder{ResponseWriter: w, status: 200}

```

```

        next.ServeHTTP(rec, r)
        logger.Info("http_request",
            "method", r.Method,
            "path", r.URL.Path,
            "status", rec.status,
            "duration_ms", time.Since(start).Milliseconds(),
            "request_id", r.Context().Value(requestIDKey),
        )
    })
}
}

// Recover: handler panic అయితే crash కాకుండా 500 (server బతికుంటుంది)
func Recover(logger *slog.Logger) Middleware {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            defer func() {
                if rec := recover(); rec != nil {
                    logger.Error("panic recovered", "err", rec,
                        "path", r.URL.Path)
                    http.Error(w, "internal error", http.StatusInternalServerError)
                }
            }()
            next.ServeHTTP(w, r)
        })
    }
}

func main() {
    logger := slog.New(slog.NewJSONHandler(os.Stdout, nil))

    mux := http.NewServeMux()
    // Go 1.22: method + path variables native
    mux.HandleFunc("GET /users/{id}", func(w http.ResponseWriter, r *http.Request) {
        id := r.PathValue("id") // path variable
        w.Write([]byte("user " + id))
    })
    mux.HandleFunc("POST /users", func(w http.ResponseWriter, r *http.Request) {
        w.WriteHeader(http.StatusCreated)
    })

    // middleware chain: Recover(RequestID(Logging(mux)))
    handler := Chain(mux,
        RequestID,
        Logging(logger),
        Recover(logger),
    )

    srv := &http.Server{
        Addr:    ":8080",
        Handler: handler,
        // ---- timeouts: production లో తప్పనిసరి (defaults = infinite = ప్రమాదం) ----
    }
}

```

```

    ReadHeaderTimeout: 3 * time.Second, // slowloris నుంచి కాపాడుతుంది
    ReadTimeout:       5 * time.Second, // request body సహా చదవడానికి max
    WriteTimeout:      10 * time.Second, // response రాయడానికి max
    IdleTimeout:       60 * time.Second, // keep-alive idle max
    MaxHeaderBytes:    1 << 20,         // 1 MB header cap
}
logger.Info("listening", "addr", srv.Addr)
srv.ListenAndServe()
}

```

Timeouts — ఏది దేని కోసం

Timeout	ఏమి కొలుస్తుంది	ఎందుకు ముఖ్యం
ReadHeaderTimeout	request headers చదవడం	slowloris attack (నెమ్మది header పంపి connection hold) నుంచి రక్షణ
ReadTimeout	headers + body పూర్తిగా చదవడం	slow client body నుంచి రక్షణ
WriteTimeout	response రాయడం	slow client response consume నుంచి రక్షణ
IdleTimeout	keep-alive idle time	idle connections leak ఆపుతుంది
http.TimeoutHandler	per-handler overall deadline	ఒక handler ఎక్కువసేపు తీసుకుంటే 503

Golden rule: `http.ListenAndServe` (bare) production లో ఎప్పుడూ వాడకు - defaults infinite timeouts. ఎప్పుడూ `&http.Server{...timeouts...}` . Per-endpoint deadline కి `context` + `http.TimeoutHandler` .

Trade-offs

- **stdlib vs framework:** stdlib చాలు simple services కి - zero dependencies, stable. Complex routing/middleware ecosystem కావాలంటే chi/gin (Topic 5).
- **WriteTimeout streaming కి సమస్య:** long SSE/streaming responses WriteTimeout దాటుతాయి - వాటికి WriteTimeout=0 పెట్టి context deadline వాడు.
- **middleware order matters:** Recover బయట (అన్నిటిని కాపాడాలి), RequestID ముందు (logging కి id కావాలి).

Key Points

- Go web = `http.Handler` interface (`ServeHTTP`). Middleware = Handler-wrapping-Handler.
- ఎప్పుడూ `http.Server{}` **explicit timeouts** - bare `ListenAndServe` కాదు.
- Go 1.22 ServeMux: `"GET /users/{id}" + r.PathValue("id")` - basic routing కి framework అవసరం లేదు.
- Middleware order: Recover (బయట) → RequestID → Logging → Auth → handler.
- `panic` recover middleware లేకపోతే ఒక్క panic మొత్తం server కూల్పోచ్చు.

Interview దృష్టి

"Go లో HTTP server ఎలా secure చేస్తావు?" → "stdlib `http.Server` explicit timeouts (`ReadHeaderTimeout` slowloris కి, `Read/Write/Idle`), `MaxHeaderBytes` cap, `panic-recover` middleware,

request-id middleware for tracing. Bare ListenAndServe ఎప్పుడూ వాడను - infinite timeouts." stdlib తో ఇంత చేయగలవని చూపించడమే Go strength.

5. Routing & middleware frameworks (chi/gin/echo) + custom middleware

వివరణ

Go 1.22 ServeMux basic routing చేస్తుంది, కానీ real apps కి కావాల్సినవి: **route groups, URL params, middleware per-group, sub-routers**. అందుకే frameworks. Go philosophy = "library over framework" - heavy magic కాదు, thin helpers.

- **chi** - stdlib-compatible (http.Handler). చాలా idiomatic, lightweight, middleware-first. Standard library తో 100% interop. చాలామంది **senior Go devs default choice**.
- **gin** - fast, popular, batteries-included (JSON binding, validation). తన సొంత gin.Context వాడుతుంది (stdlib కాదు).
- **echo** - gin లాంటిదే, clean API, built-in middleware ఎక్కువ.
- **fiber** - Express-like, fasthttp మీద (stdlib net/http కాదు - ecosystem incompatibility risk).

Real-life Scenario

chi = నీ ఇంటి plumbing కి standard fittings. ఏ pipe అయినా (stdlib handler) సరిపోతుంది, replace easy.
gin = అన్నీ కలిపి వచ్చే modular kitchen - fast to install, అన్నీ ఒకేచోట, కానీ దాని fittings (gin.Context) దానివే - standard pipes సరిపోవు. Migration/interop కావాలంటే chi; fast delivery + built-in features కావాలంటే gin.

Code — chi router + groups + middleware

```
package main

import (
    "net/http"
    "time"

    "github.com/go-chi/chi/v5"
    "github.com/go-chi/chi/v5/middleware"
)

func main() {
    r := chi.NewRouter()

    // built-in middleware (order matters - పైవి ముందు)
    r.Use(middleware.RequestID)           // request-id
    r.Use(middleware.RealIP)              // X-Forwarded-For నుంచి real IP
    r.Use(middleware.Logger)              // request logging
    r.Use(middleware.Recoverer)           // panic → 500
    r.Use(middleware.Timeout(10 * time.Second)) // per-request deadline (context)

    // public routes
    r.Get("/health", func(w http.ResponseWriter, r *http.Request) {
        w.Write([]byte("ok"))
    })
}
```

```

// versioned API group (sub-router)
r.Route("/api/v1", func(r chi.Router) {

    r.Route("/users", func(r chi.Router) {
        r.Get("/", listUsers)           // GET  /api/v1/users
        r.Post("/", createUser)         // POST /api/v1/users
        r.Get("/{id}", getUser)         // GET  /api/v1/users/123
    })

    // protected group - auth middleware ఈ group కి మాత్రమే
    r.Group(func(r chi.Router) {
        r.Use(AuthMiddleware) // custom (కీద)
        r.Delete("/users/{id}", deleteUser)
        r.Post("/admin/reindex", reindex)
    })
})

http.ListenAndServe(":8080", r) // (ఉదాహరణకి; prod లో http.Server{} timeouts)
}

func getUser(w http.ResponseWriter, r *http.Request) {
    id := chi.URLParam(r, "id") // chi param extraction
    w.Write([]byte("user " + id))
}

func listUsers(w http.ResponseWriter, r *http.Request) {}
func createUser(w http.ResponseWriter, r *http.Request) {}
func deleteUser(w http.ResponseWriter, r *http.Request) {}
func reindex(w http.ResponseWriter, r *http.Request) {}

// ---- custom auth middleware (stdlib-compatible → chi తో సరిపోతుంది) ----
func AuthMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        token := r.Header.Get("Authorization")
        if token == "" {
            http.Error(w, "unauthorized", http.StatusUnauthorized)
            return // chain ఇక్కడే ఆగుతుంది (next పిలవలేదు)
        }
        // ...token validate, user ని context లో పెట్టు...
        next.ServeHTTP(w, r)
    })
}

```

Framework పోలిక

అంశం	chi	gin	echo	stdlib (1.22)
stdlib http.Handler compatible	✅ 100%	❌ (gin.Context)	❌ (echo.Context)	✅

Route groups / sub-routers	✓	✓	✓	✗ (manual)
URL params	✓	✓	✓	✓ (1.22+)
Built-in JSON bind/validate	✗ (manual)	✓	✓	✗
Middleware ecosystem	పెద్దది	పెద్దది	పెద్దది	చిన్నది
Performance	high	very high	very high	high
Learning/lock-in	తక్కువ	మధ్యం	మధ్యం	తక్కువ
Best fit	idiomatic, interop	rapid API dev	rapid, clean	minimal deps

Trade-offs

- **chi**: stdlib interop, easy testing, minimal magic - కానీ JSON binding/validation నువ్వే రాయాలి.
- **gin/echo**: fast to build (binding, validation built-in) - కానీ `gin.Context` lock-in, stdlib middleware directly పనిచేయదు (adapter కావాలి).
- **fiber**: వేగం ఎక్కువ (fasthttp) కానీ `net/http` ecosystem (many libs, HTTP/2, tooling) miss అవుతావ్. జాగ్రత్త.

Key Points

- Go = "library over framework". chi = idiomatic (stdlib-compatible), gin/echo = batteries-included.
- Middleware = `func(http.Handler) http.Handler` (chi/stdlib). Order matters.
- Route groups తో per-group middleware (public vs auth-protected).
- stdlib-compatible middleware ఎక్కువ reusable - chi ఎంచుకుంటే lock-in తక్కువ.

Interview దృష్టి

"ఏ router వాడతావ్, ఎందుకు?" → "chi - stdlib `http.Handler` compatible కాబట్టి existing middleware/testing tools పనిచేస్తాయి, lock-in తక్కువ, idiomatic. Rapid CRUD + built-in validation కావాలంటే gin. fiber avoid చేస్తా - fasthttp మీద, net/http ecosystem miss అవుతా." Framework choice ని trade-off గా చెప్పడం seniority signal.

6. gRPC in Go — protobuf, unary & streaming, interceptors

వివరణ

gRPC = Google RPC framework - **HTTP/2 + Protocol Buffers (protobuf)** మీద. Service-to-service (internal microservices) communication కి REST కంటే వేగం, type-safety ఎక్కువ. Go gRPC first-class (Google Go లో రాశారు).

- **protobuf** - `.proto` file లో service + messages define చేస్తావ్. `protoc` code generate చేస్తుంది (Go structs + client/server stubs). **schema-first, strongly typed, binary encoding** (JSON కంటే చిన్నది, fast).
- **HTTP/2** - multiplexing (ఒక్క connection మీద అనేక streams), header compression, bidirectional streaming.
- **4 RPC types**: unary (1 req → 1 resp), server-streaming, client-streaming, bidirectional streaming.
- **interceptors** - middleware equivalent (unary + stream).

Real-life Scenario

REST = postcard (JSON, ఎవరైనా చదవగలరు, పెద్దది). gRPC = ద్వితీయంలో typed telegram (protobuf, compact, sender/receiver ఇద్దరికీ format ముందే తెలుసు). Internal services ఒకదానితో ఒకటి చాలా సార్లు మాట్లాడతాయి - ప్రతిసారీ postcard (verbose JSON, parse cost) బదులు, ముందే agreed short-code (protobuf schema) వాడితే వేగం, తప్పులు తక్కువ. కానీ browser కి telegram చదవడం రాదు (gRPC browser-native కాదు) - అక్కడ postcard (REST/JSON) వాడాలి.

Code — proto + Go server/client

```
// api/proto/order.proto
syntax = "proto3";
package order.v1;
option go_package = "myservice/gen/orderpb";

service OrderService {
    rpc GetOrder(GetOrderRequest) returns (Order);           // unary
    rpc WatchOrders(WatchRequest) returns (stream Order);    // server-streaming
}

message GetOrderRequest { string id = 1; }
message WatchRequest    { string user_id = 1; }
message Order {
    string id = 1;
    string user_id = 2;
    int64 amount_cents = 3;
    string status = 4;
}
```

```
// server side
package main

import (
    "context"
    "log"
    "net"
    "time"

    "google.golang.org/grpc"
    "google.golang.org/grpc/codes"
    "google.golang.org/grpc/status"

    pb "myservice/gen/orderpb"
)

type orderServer struct {
    pb.UnimplementedOrderServiceServer // forward-compat (కొత్త methods add అయినా
    break కాదు)
}
```

```

// unary RPC
func (s *orderServer) GetOrder(ctx context.Context, req *pb.GetOrderRequest)
(*pb.Order, error) {
    if req.Id == "" {
        // typed gRPC error codes (HTTP status కీ బదులు)
        return nil, status.Error(codes.InvalidArgument, "id required")
    }
    // context deadline honor చేయి (client timeout propagate అవుతుంది)
    select {
    case <-ctx.Done():
        return nil, status.Error(codes.DeadlineExceeded, ctx.Err().Error())
    default:
    }
    return &pb.Order{Id: req.Id, UserId: "u1", AmountCents: 4999, Status: "PAID"},
    nil
}

// server-streaming RPC: server అనేక messages పంపుతుంది
func (s *orderServer) WatchOrders(req *pb.WatchRequest, stream
pb.OrderService_WatchOrdersServer) error {
    for i := 0; i < 3; i++ {
        if err := stream.Context().Err(); err != nil {
            return err // client disconnect / deadline
        }
        if err := stream.Send(&pb.Order{Id: "o", UserId: req.UserId, Status:
"NEW"}); err != nil {
            return err
        }
        time.Sleep(time.Second)
    }
    return nil
}

// ---- unary interceptor = middleware (logging, auth, metrics) ----
func loggingInterceptor(ctx context.Context, req any,
info *grpc.UnaryServerInfo, handler grpc.UnaryHandler) (any, error) {
    start := time.Now()
    resp, err := handler(ctx, req) // actual RPC
    log.Printf("rpc=%s dur=%s err=%v", info.FullMethod, time.Since(start), err)
    return resp, err
}

func main() {
    lis, _ := net.Listen("tcp", ":9090")
    srv := grpc.NewServer(
        grpc.ChainUnaryInterceptor(loggingInterceptor), // interceptors chain
    )
    pb.RegisterOrderServiceServer(srv, &orderServer{})
    log.Println("gRPC on :9090")
    srv.Serve(lis)
}

```

```
// client side (typed, generated)
conn, _ := grpc.NewClient("localhost:9090",
    grpc.WithTransportCredentials(insecure.NewCredentials()))
defer conn.Close()

client := pb.NewOrderServiceClient(conn)

ctx, cancel := context.WithTimeout(context.Background(), 2*time.Second)
defer cancel()

order, err := client.GetOrder(ctx, &pb.GetOrderRequest{Id: "o123"}) // type-safe
call
```

gRPC vs REST — ఎప్పుడు ఏది

అంశం	gRPC	REST/JSON
Encoding	protobuf (binary, compact)	JSON (text, పెద్దది)
Transport	HTTP/2 (multiplexing, streaming)	HTTP/1.1 (సాధారణంగా)
Schema	.proto (strong, code-gen)	OpenAPI (optional)
Streaming	✅ (4 types, bidi)	❌ (SSE/WS workaround)
Browser support	❌ (grpc-web అవసరం)	✅ native
Human-readable	❌	✅ (curl, debug easy)
Best fit	internal service-to-service , low latency, streaming	public APIs , browsers, simple integration

Trade-offs

- ✅ **gRPC**: fast (binary + HTTP/2), typed contracts, streaming, deadline/cancellation propagation, load balancing built-in.
- ❌ **gRPC**: browser కి direct కాదు (grpc-web proxy), debugging harder (binary), `.proto` tooling setup, versioning discipline కావాలి.
- REST at edge, gRPC internal** - common pattern (API gateway REST తీసుకుని gRPC కి translate).

Key Points

- gRPC = HTTP/2 + protobuf. Internal microservices కి REST కంటే fast + typed.
- 4 RPC types: unary, server-stream, client-stream, bidi-stream.
- Interceptors** = middleware (unary + stream). Logging, auth, metrics ఇక్కడ.
- `context` deadline client → server automatic propagate అవుతుంది (cancellation కూడా).
- `status.Error(codes.X, ...)` - gRPC typed error codes (HTTP status కాదు).
- `UnimplementedXServer` embed చేయి - forward compatibility.

Interview దృష్టి

"REST vs gRPC ఎప్పుడు?" → "Public/browser-facing = REST (readable, native support). Internal service-to-service high-throughput/low-latency = gRPC (binary protobuf, HTTP/2 multiplexing, streaming, typed contracts, deadline propagation). Edge లో REST gateway → internal gRPC common. gRPC browser కి grpc-web proxy అవసరం."

7. API design in Go — REST vs gRPC vs GraphQL, versioning, pagination, idempotency

వివరణ

Good API = **predictable, evolvable, safe to retry**. HLD interview లో API design ఒక step. Go లో ఈ patterns ఎలా implement చేస్తామో చూద్దాం.

- **Versioning** - client break కాకుండా API మార్చడం. URL (/v1/), header, లేదా gRPC package (order.v1).
- **Pagination** - పెద్ద lists ని chunks లో. **Offset** (page number - simple కానీ deep pages slow, drift) vs **Cursor/keyset** (last-seen id - scalable, stable).
- **Idempotency** - same request రెండుసార్లు వచ్చినా side-effect ఒక్కసారే (network retry safe).
Idempotency-Key header + dedup store.

Real-life Scenario

Idempotency key = ATM withdrawal token. నువ్వు ₹5000 withdraw చేస్తుంటే network పోయి "success వచ్చిందా?" తెలియకపోతే retry చేస్తావ్. ప్రతి attempt కి **unique token** ఉంటే, bank "ఈ token ఇప్పటికే process అయింది - మళ్ళీ డబ్బు తీయను, పాత result ఇస్తా" అంటుంది. అందుకే ఎన్నిసార్లు retry చేసినా ₹5000 ఒక్కసారే withdraw. **Cursor pagination = పుస్తకంలో bookmark** - page number (offset) కాదు, "ఎక్కడ ఆపావో అక్కడి నుంచి" - మధ్యలో pages add/remove అయినా bookmark సరిగ్గా పనిచేస్తుంది.

Code — cursor pagination + idempotency middleware

```
// ---- Cursor pagination (keyset - scalable) ----
// GET /orders?limit=20&cursor=<last_id>
func listOrders(db *sql.DB) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        limit := parseLimit(r.URL.Query().Get("limit"), 20, 100) // default 20, max 100
        cursor := r.URL.Query().Get("cursor") // last seen id ("") = start

        // WHERE id > cursor ORDER BY id LIMIT n → offset కాదు, index seek (fast)
        rows, err := db.QueryContext(r.Context(),
            `SELECT id, amount FROM orders WHERE id > $1 ORDER BY id LIMIT $2`,
            cursor, limit+1) // limit+1 → "next page ఉందా" తెలుసుకోవడానికి
        if err != nil {
            http.Error(w, "db error", 500)
            return
        }
        defer rows.Close()

        var orders []Order
        for rows.Next() {
```

```

        var o Order
        rows.Scan(&o.ID, &o.Amount)
        orders = append(orders, o)
    }
    var nextCursor string
    if len(orders) > limit { // limit+1 వచ్చింది → ఇంకా pages ఉన్నాయి
        nextCursor = orders[limit-1].ID
        orders = orders[:limit]
    }
    writeJSON(w, map[string]any{"data": orders, "next_cursor": nextCursor})
}
}

// ---- Idempotency middleware (retry-safe writes) ----
type IdempotencyStore interface {
    Get(ctx context.Context, key string) ([]byte, bool, error)
    Set(ctx context.Context, key string, resp []byte, ttl time.Duration) error
}

func Idempotency(store IdempotencyStore) Middleware {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            // POST/PUT/PATCH లాంటి unsafe methods కి మాత్రమే
            if r.Method == http.MethodGet {
                next.ServeHTTP(w, r)
                return
            }
            key := r.Header.Get("Idempotency-Key")
            if key == "" {
                next.ServeHTTP(w, r) // key లేకపోతే normal (కొదరు 400 return చేస్తారు)
                return
            }
            // ఇప్పటికీ process అయిందా? → cached response తిరిగి ఇవ్వు
            if cached, ok, _ := store.Get(r.Context(), key); ok {
                w.Header().Set("Idempotent-Replay", "true")
                w.Write(cached)
                return
            }
            // కొత్తది - process చేసి, response capture చేసి, store చేయి
            rec := &bodyRecorder{ResponseWriter: w}
            next.ServeHTTP(rec, r)
            if rec.status < 500 { // success/client-error cache; server error కాదు
                (retry చేయనివ్వు)
                store.Set(r.Context(), key, rec.body.Bytes(), 24*time.Hour)
            }
        })
    }
}

```

REST vs gRPC vs GraphQL — పోలిక

అంశం	REST	gRPC	GraphQL
Data fetching	fixed endpoints (over/under-fetch)	fixed RPCs	client asks exact fields
Encoding	JSON	protobuf (binary)	JSON
Streaming	✗ (SSE/WS)	✓	subscriptions
Best fit	public APIs, CRUD	internal services	complex client-driven UIs, BFF
Caching	HTTP cache easy	harder	harder (POST)
Go libs	stdlib, chi	google.golang.org/grpc	gqlgen
Over/under-fetch	సమస్య	schema-bound	పరిష్కారం
Complexity	తక్కువ	మధ్యం	ఎక్కువ (N+1, resolvers)

Versioning strategies

Strategy	ఉదాహరణ	Trade-off
URL path	/api/v1/users	simple, visible, caching-friendly (అత్యంత common)
Header	Accept: application/vnd.api.v2+json	clean URLs కానీ hidden
gRPC package	order.v1, order.v2	proto-native, parallel versions
Query param	?version=2	discouraged (caching సమస్య)

Trade-offs

- **Offset vs cursor:** offset simple (page 5 కి direct jump) కానీ deep pages slow (DB అన్ని rows skip చేయాలి) + insert అయితే drift (duplicate/skip rows). Cursor scalable + stable కానీ random page jump కుదరదు.
- **Idempotency store:** Redis (TTL, fast) common. Response cache TTL జాగ్రత్త - too long = stale, too short = replay window చిన్నది.
- **GraphQL power vs complexity:** flexible fetching కానీ N+1 queries, caching hard, rate-limiting hard (query cost). Public API కి REST simpler.

Key Points

- Versioning: URL path (/v1/) most common. **Never break existing clients** - additive changes preferred.
- **Cursor (keyset) pagination** > offset for scale (index seek, no drift). limit+1 trick తో "next page ఉందా" తెలుసుకో.
- **Idempotency-Key** header + dedup store (Redis) → retries safe. Server errors (5xx) cache చేయకు (retry allow చేయాలి).
- REST public, gRPC internal, GraphQL client-driven complex UIs.

Interview దృష్టి

"Payment API retry-safe ఎలా?" → "Idempotency-Key header + Redis dedup store. మొదటిసారి process చేసి result store; retry వస్తే stored result తిరిగి ఇస్తా - double charge కాదు. 5xx errors cache చేయను (retry allow). Pagination cursor-based - index seek, drift లేదు." ఇది real production maturity చూపిస్తుంది.

8. Realtime in Go — WebSockets, SSE, long polling

వివరణ

Client కి server నుంచి **push** కావాలంటే (chat, notifications, live scores, dashboards) - normal request/response సరిపోదు. మూడు techniques:

- **Long polling** - client request పంపి, server data వచ్చేదాకా hold చేస్తుంది (కొత్త data వస్తే respond, లేకపోతే timeout → client మళ్ళీ). Simple, HTTP-only. Go goroutines దీనికి బాగా సరిపోతాయి (blocked goroutine cheap).
- **SSE (Server-Sent Events)** - ఒక్క long-lived HTTP connection మీద server **unidirectional** stream (server → client). Text-only, auto-reconnect, HTTP-native. Notifications/feed కి perfect.
- **WebSockets** - ఒక్క connection మీద **bidirectional** full-duplex. Chat, gaming, collaborative editing. HTTP upgrade తో మొదలు. Go libs: `gorilla/websocket`, `coder/websocket` (nhooyr).

Go అనుకూలం: ఒక్క connection కి ఒక cheap goroutine → లక్షల concurrent connections ఒక్క server మీద (thread-per-connection Java లో ఖరీదు).

Real-life Scenario

Long polling = "పార్సెల్ వచ్చిందా?" అని పోస్టాఫీసుకు ఫోన్ చేసి line లో ఉండటం - వస్తే చెప్తారు, రాకపోతే కొంతసేపటికి "ఇంకా రాలేదు, మళ్ళీ చేయి" అంటారు. **SSE = news channel** - నువ్వు channel పెట్టుకుంటే వాళ్ళు updates చెప్తూ ఉంటారు (ఒకవైపు - నువ్వు వాళ్ళతో మాట్లాడవు). **WebSocket = phone call** - ఇద్దరూ ఏకకాలంలో మాట్లాడుకోవచ్చు (bidirectional), line open గా ఉంటుంది.

Code — SSE + WebSocket

```
// ---- SSE: server → client unidirectional stream ----
func sseHandler(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Content-Type", "text/event-stream")
    w.Header().Set("Cache-Control", "no-cache")
    w.Header().Set("Connection", "keep-alive")

    flusher, ok := w.(http.Flusher) // chunk-by-chunk పంపడానికి flush అవసరం
    if !ok {
        http.Error(w, "streaming unsupported", 500)
        return
    }

    ticker := time.NewTicker(time.Second)
    defer ticker.Stop()

    for {
        select {
        case <-r.Context().Done(): // client disconnect → goroutine leak అవు
            return
        case t := <-ticker.C:
```

```

        // SSE format: "data: <payload>\n\n"
        fmt.Fprintf(w, "data: {\"time\": \"%s\"}\n\n", t.Format(time.RFC3339))
        flusher.Flush() // వెంటనే client కి పంపు (buffer లో ఆగకుండా)
    }
}

// ---- WebSocket: bidirectional (coder/websocket) ----
import "github.com/coder/websocket"

func wsHandler(w http.ResponseWriter, r *http.Request) {
    conn, err := websocket.Accept(w, r, nil) // HTTP → WS upgrade
    if err != nil {
        return
    }
    defer conn.Close(websocket.StatusInternalError, "closing")

    ctx := r.Context()
    for {
        // read (client → server)
        _, data, err := conn.Read(ctx)
        if err != nil {
            return // client closed / error
        }
        // echo back (server → client)
        if err := conn.Write(ctx, websocket.MessageText, data); err != nil {
            return
        }
    }
}

```

Realtime techniques — పోలిక

అంశం	Long Polling	SSE	WebSocket
Direction	server → client (req మీద)	server → client	bidirectional
Protocol	HTTP	HTTP	ws:// (HTTP upgrade)
Connection	short (repeated)	long-lived	long-lived
Auto-reconnect	manual	✅ built-in (browser)	manual
Binary support	✅	❌ (text only)	✅
Overhead	ఎక్కువ (repeated requests)	తక్కువ	తక్కువ
Firewall/proxy friendly	✅	✅	కొన్నిసార్లు block

Best fit	legacy, simple	notifications, feeds, dashboards	chat, gaming, collab
Go lib	stdlib	stdlib	gorilla / coder

Go concurrency at connection scale

```

Client 1 —ws—|
Client 2 —ws—|   ఒక్క connection → ఒక goroutine
Client 3 —ws—|   (blocked goroutine ~2KB, cheap)
...
Client 500K —ws—|
                |
Go scheduler: లక్షల goroutines → కొన్ని OS threads
(thread-per-connection అయితే 500K threads = అసాధ్యం)

```

కీలకం: ప్రతి long-lived connection కి **client disconnect అయితే goroutine అగాలి** - లేకపోతే **goroutine leak** (memory పెరుగుతూ పోతుంది). ఎప్పుడూ `r.Context().Done()` / `ctx watch` చేయి.

Trade-offs

- **Long polling:** simple, universal కానీ inefficient (repeated connections, latency). కొత్త systems లో SSE/WS preferred.
- **SSE:** simple, HTTP-native, auto-reconnect కానీ server→client మాత్రమే + text-only + browser కి per-domain connection limit (HTTP/1.1). Notifications కి ideal.
- **WebSocket:** powerful (bidirectional, binary) కానీ stateful (connection scaling కష్టం - sticky sessions / pub-sub backplane like Redis అవసరం), some proxies block, reconnect logic సువ్వే.
- **Scaling stateful connections:** connections ఒక server లో ఉంటాయి → horizontal scale కి **Redis pub/sub / Kafka backplane** తో fan-out (Topic 12).

Key Points

- Long poll → SSE → WebSocket: complexity పెరుగుతూ, capability పెరుగుతుంది.
- SSE = server-push notifications/feeds (unidirectional, auto-reconnect). WS = bidirectional (chat/games).
- Go goroutine-per-connection cheap → లక్షల concurrent connections. కానీ **disconnect అయితే goroutine అవు** (context watch) - leak జాగ్రత్త.
- SSE: `text/event-stream` + `flusher.Flush()`. WS: HTTP upgrade + read/write loop.
- Stateful WS horizontal scale కి pub/sub backplane (Redis/Kafka) అవసరం.

Interview దృష్టి

"1M concurrent chat users Go లో ఎలా?" → "WebSocket, connection-per-goroutine (cheap - 2KB). Single server ~100K-500K connections; scale కి multiple servers + Redis pub/sub backplane (ఒక server కి వచ్చిన message ని అన్ని servers కి fan-out). Sticky routing లేదా shared presence store. Disconnect అయితే goroutine + context cleanup - leak avoid." Go's connection scalability + backplane pattern చెప్పడం key.

Part 3 — Data & Caching Layer

9. database/sql deep — pool tuning, prepared statements, context

వివరణ

Go stdlib `database/sql` = SQL DB access కి **generic interface + connection pool**. Actual DB driver వేరుగా import చేస్తావ్ (pgx for Postgres, mysql). ముఖ్యమైన విషయం: `*sql.DB` **ఒక్క connection కాదు - ఒక pool** (thread-safe, long-lived, share చేయాలి). ప్రతి request కి కొత్త `sql.Open` చేయకూడదు - app జీవితకాలం ఒక్కసారి create చేసి share చేయాలి.

Pool tuning production లో అత్యంత ముఖ్యం - misconfig అయితే DB connections exhaust అవుతాయి లేదా idle waste. మూడు knobs:

- **SetMaxOpenConns(n)** - మొత్తం max connections (idle + in-use). DB `max_connections` కంటే తక్కువ ఉండాలి (అన్ని app instances కలిపి).
- **SetMaxIdleConns(n)** - pool లో ready-to-reuse idle connections. తక్కువ = ప్రతిసారి కొత్త connection (slow). MaxOpen తో సమానంగా పెట్టడం common.
- **SetConnMaxLifetime(d)** - ఒక connection ఎంతసేపు live. DB-side timeouts / LB rebalancing / stale connections కి. (ఉదా 30 min).
- **SetConnMaxIdleTime(d)** - idle connection ఎంతసేపు pool లో.

Context - ప్రతి query కి `QueryContext` / `ExecContext` వాడి request context pass చేయి → request cancel/timeout అయితే query కూడా cancel (DB కి signal). `Query` (context లేని) production లో వాడకు.

Real-life Scenario

Connection pool = restaurant లో waiters. DB కి connect చేయడం = కొత్త waiter ని hire+train చేయడం (ఖరీదు, నెమ్మది). అందుకే fixed సంఖ్య waiters (`MaxOpenConns`) pre-hire చేసి, table (query) వచ్చినప్పుడు free waiter ని assign చేస్తాం. అందరూ busy అయితే కొత్త customer wait చేస్తాడు (pool exhausted). `MaxIdleConns` = idle గా ready ఉన్న waiters (వెంటనే serve). `ConnMaxLifetime` = waiter shift time (అలసిపోయిన/stale waiter ని replace). Waiters ఎక్కువ hire చేస్తే (`MaxOpen` too high) restaurant (DB) కి జీతాలు (resources) భారం - crash.

Code — pool setup + context queries

```
package store

import (
    "context"
    "database/sql"
    "fmt"
    "time"

    _ "github.com/jackc/pgx/v5/stdlib" // Postgres driver (blank import → register)
)

// NewDB = app జీవితకాలం ఒక్కసారి. *sql.DB share చేయి (pool, thread-safe).
func NewDB(dsn string, maxOpen int) (*sql.DB, error) {
    db, err := sql.Open("pgx", dsn) // ఇక్కడ connect కాదు - lazy (మొదటి query కి)
    if err != nil {
        return nil, err
    }
    // ---- pool tuning (production critical) ----
    db.SetMaxOpenConns(maxOpen) // మొత్తం max (DB max_connections /
    instances)
```

```

db.SetMaxIdleConns(maxOpen) // idle కూడా same → reuse ఎక్కువ (churn
తక్కువ)
db.SetConnMaxLifetime(30 * time.Minute) // stale/LB rebalance కి
db.SetConnMaxIdleTime(5 * time.Minute) // idle conn ని free చేయి

// startup లోనే connectivity verify (fail-fast)
ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
defer cancel()
if err := db.PingContext(ctx); err != nil {
    return nil, fmt.Errorf("db ping: %w", err)
}
return db, nil
}

type User struct {
    ID    string
    Email string
}

// context-aware query: request timeout/cancel → query కూడా cancel అవుతుంది
func GetUser(ctx context.Context, db *sql.DB, id string) (*User, error) {
    var u User
    // QueryRowContext + parameterized ($1) → SQL injection safe
    err := db.QueryRowContext(ctx,
        `SELECT id, email FROM users WHERE id = $1`, id).Scan(&u.ID, &u.Email)
    if err == sql.ErrNoRows {
        return nil, fmt.Errorf("user not found")
    }
    if err != nil {
        return nil, fmt.Errorf("query user: %w", err)
    }
    return &u, nil
}

// transaction: money-like ops (atomicity). defer Rollback safety net.
func Transfer(ctx context.Context, db *sql.DB, from, to string, cents int64) error {
    tx, err := db.BeginTx(ctx, &sql.TxOptions{Isolation: sql.LevelSerializable})
    if err != nil {
        return err
    }
    defer tx.Rollback() // Commit అయితే no-op; ఏదైనా fail అయితే rollback

    if _, err = tx.ExecContext(ctx,
        `UPDATE accounts SET balance = balance - $1 WHERE id = $2`, cents, from);
    err != nil {
        return err
    }
    if _, err = tx.ExecContext(ctx,
        `UPDATE accounts SET balance = balance + $1 WHERE id = $2`, cents, to); err
    != nil {
        return err
    }
}

```

```
    return tx.Commit()
}
```

Rows iteration — leak జాగ్రత్త

```
rows, err := db.QueryContext(ctx, `SELECT id, email FROM users LIMIT 100`)
if err != nil {
    return err
}
defer rows.Close() // * తప్పనిసరి - లేకపోతే connection leak (pool exhaust)

var users []User
for rows.Next() {
    var u User
    if err := rows.Scan(&u.ID, &u.Email); err != nil {
        return err
    }
    users = append(users, u)
}
return rows.Err() // loop లో error వచ్చిందా check (ముఖ్యం - మర్చిపోతారు)
```

Pool settings — cheat sheet

Setting	ఏమి చేస్తుంది	Guidance
SetMaxOpenConns	total max conns	DB max_connections / (instances × 1.2) కంటే తక్కువ
SetMaxIdleConns	idle ready conns	MaxOpen తో సమానం (reuse ఎక్కువ, churn తక్కువ)
SetConnMaxLifetime	conn max age	30 min (LB/DNS rebalance, stale conns)
SetConnMaxIdleTime	idle max age	5 min (idle waste తగ్గించు)

లెక్క: DB max_connections = 100, app 4 instances అయితే - ఒక్కో instance MaxOpenConns ~20 (80 total, buffer వదిలి). MaxOpen too high → "too many connections" DB error. Too low → requests wait (latency).

Trade-offs

- sql.DB vs pgxpool:** stdlib database/sql generic, driver-agnostic. Postgres-specific features (COPY, LISTEN/NOTIFY, better perf) కావాలంటే pgxpool (native pgx pool) - కానీ database/sql interface వదులుకుంటావ్.
- Prepared statements:** repeated queries కి fast (parse ఒక్కసారి) కానీ database/sql prepared statement ఒక connection కి bound - pool తో careful. చాలా drivers auto-prepare చేస్తాయి.
- MaxOpenConns too high:** DB overwhelm (connections కూడా DB memory తింటాయి). Connection pooler (PgBouncer) middle layer common at scale.

Key Points

- *sql.DB = pool, ఒక్కసారి create చేసి share చేయి (ప్రతి request కి sql.Open కాదు).
- Pool tune చేయి:** MaxOpenConns (DB limit కంటే తక్కువ), MaxIdleConns, ConnMaxLifetime. Misconfig = production outage.

- ఎప్పుడూ `...Context` variants (`QueryContext` , `ExecContext`) + parameterized queries (\$1 - injection safe).
- `rows.Close()` (defer) + `rows.Err()` తప్పనిసరి - లేకపోతే connection leak.
- Transactions: `BeginTx` + `defer tx.Rollback()` (safety) + `tx.Commit()` .

Interview దృష్టి

"Go service DB connections exhaust అవుతున్నాయి - ఎలా debug?" → "MaxOpenConns అన్ని instances కలిపి DB max_connections దాటుతుందేమో - లెక్క వేస్తా. `rows.Close()` / `tx.Rollback()` leak ఉందేమో (missing defer). ConnMaxLifetime stale conns pile చేస్తుందేమో. `db.Stats()` (WaitCount, InUse) metrics చూస్తా. PgBouncer pooler add చేయొచ్చు scale కీ." Pool math + leak awareness senior signal.

10. ORMs & query builders (GORM, sqlx, sqlc)

వివరణ

Raw `database/sql` boilerplate ఎక్కువ (Scan ప్రతి field). దీన్ని తగ్గించడానికి tools:

- **GORM** - full ORM (Object-Relational Mapping). Structs ↔ tables auto, associations, migrations, hooks. Rapid dev కానీ "magic", generated SQL అపారదర్శకం, reflection overhead, N+1 traps.
- **sqlx** - `database/sql` కి thin extension. `StructScan` (rows → struct auto), named params. **SQL నువ్వే రాస్తావ్** - control + తక్కువ boilerplate. Middle ground.
- **sqlc** - **SQL నుంచి Go code generate చేస్తుంది**. `.sql` files రాస్తావ్ → `sqlc` type-safe Go functions generate చేస్తుంది. **Compile-time safety + raw SQL performance + zero runtime reflection**. Senior Go community favorite.

Real-life Scenario

GORM = auto-translate app (Google Translate). నువ్వు తెలుగులో (Go structs) చెప్తే అదే SQL లోకి translate చేస్తుంది - fast కానీ కొన్నిసార్లు awkward/inefficient అనువాదం (bad SQL), నువ్వు చూడవు. **sqlx = dictionary తో నువ్వే translate** - నీ SQL, `sqlx` కేవలం Scan boilerplate తగ్గిస్తుంది. **sqlc = professional translator ముందే document translate చేసి ఇస్తుంది** - నువ్వు SQL రాస్తే, `sqlc` దాన్ని typed Go గా pre-generate చేస్తుంది, compile time లోనే తప్పులు పట్టుకుంటుంది.

Code — same query, 3 ways

```
// ---- 1. GORM (ORM - magic ఎక్కువ) ----
type User struct {
    ID      uint   `gorm:"primaryKey"`
    Email   string `gorm:"uniqueIndex"`
    Name    string
}

var user User
db.Where("email = ?", "a@b.com").First(&user) // SQL auto-generate
db.Create(&User{Email: "x@y.com", Name: "X"}) // INSERT auto
// N+1 trap: db.Find(&users); loop లో user.Orders access → per-user query!
// fix: db.Preload("Orders").Find(&users)

// ---- 2. sqlx (thin - నీ SQL + auto Scan) ----
type UserRow struct {
```

```

    ID    string `db:"id"`
    Email string `db:"email"`
}

var u UserRow
// StructScan: row → struct auto (manual Scan అవసరం లేదు)
err := sqlxDB.GetContext(ctx, &u,
    `SELECT id, email FROM users WHERE email = $1`, "a@b.com")

var users []UserRow
sqlxDB.SelectContext(ctx, &users, `SELECT id, email FROM users LIMIT 10`)

// ---- 3. sqlc (SQL → generated type-safe Go) ----
// query.sql లో:
// -- name: GetUserByEmail :one
// SELECT id, email FROM users WHERE email = $1;
//
// sqlc generate → ఈ typed function auto-create:
u2, err := queries.GetUserByEmail(ctx, "a@b.com")
// u2 typed struct; column mismatch = COMPILE error (runtime కాదు!)

```

ORM/tool పోలిక

అంశం	GORM	sqlx	sqlc	raw database/sql
SQL ఎవరు రాస్తారు	GORM (auto)	నువ్వు	నువ్వు (.sql)	నువ్వు
Type safety	runtime	runtime	compile-time	runtime
Boilerplate	తక్కువ	మధ్యం	తక్కువ	ఎక్కువ
SQL control	తక్కువ (opaque)	పూర్తి	పూర్తి	పూర్తి
Performance	reflection overhead	good	best (no reflection)	best
Migrations/hooks	built-in	✗	✗	✗
Learning curve	పెద్దది (magic)	చిన్నది	చిన్నది	-
N+1 risk	ఎక్కువ (associations)	తక్కువ	తక్కువ	తక్కువ
Best fit	rapid CRUD, prototypes	control + convenience	type-safe production	full control

Trade-offs

- GORM:** fast to start, associations/migrations/hooks free - కానీ generated SQL అపారదర్శకం (perf debugging కష్టం), N+1 traps, reflection cost, complex queries awkward. Large/perf-critical services లో చాలామంది avoid.

- **sqlx**: SQL control + boilerplate తగ్గింపు - కానీ compile-time safety లేదు (typo → runtime error), migrations నువ్వే.
- **sqlc: compile-time safety** (column rename → build fail), best perf, no magic - కానీ dynamic queries (variable WHERE clauses) awkward, code-gen step CI లో.

Key Points

- GORM = full ORM (rapid, magic, perf/N+1 traps). sqlx = thin (నీ SQL + auto scan). sqlc = **SQL → type-safe generated Go**.
- **sqlc = senior favorite**: compile-time safety + raw SQL perf + zero reflection.
- GORM లో N+1 జాగ్రత్త - Preload వాడు.
- Migrations కి separate tool (`golang-migrate` , `goose`) - sqlc/sqlx migrations చేయవు.

Interview దృష్టి

"GORM వాడతావా?" → "Rapid prototype/CRUD కి సరే. Perf-critical / large service కి sqlc prefer చేస్తా - compile-time type safety (column rename చేస్తే build fail), raw SQL perf, no reflection overhead, generated SQL transparent. GORM లో opaque SQL + N+1 traps debugging కష్టం." Tool ని dogma కాకుండా trade-off గా చెప్పు.

11. Caching in Go — in-process, Redis, singleflight

వివరణ

Cache = costly operation (DB query, computation) result ని fast store లో పెట్టి, మళ్ళీ అడిగితే అక్కడి నుంచి.

Read-heavy systems కి single biggest lever. Go లో రెండు levels:

- **In-process cache** - same process memory (`map` + mutex, `sync.Map` , LRU library like `hashicorp/golang-lru`). అత్యంత fast (network లేదు) కానీ **per-instance** (each pod వేరు → consistency సమస్య), memory-bound, restart అయితే పోతుంది.
- **Distributed cache (Redis)** - external, అన్ని instances share చేస్తాయి. `go-redis` client. Network hop (fast కానీ in-process కంటే slow), కానీ shared + persistent (optional) + big.

Caching patterns:

- **Cache-aside (lazy)** - app cache చూస్తుంది; miss అయితే DB నుంచి తెచ్చి cache లో పెడుతుంది. Most common.
- **Write-through** - write DB + cache రెండింటికీ (cache ఎప్పుడూ fresh, write slow).
- **Write-back** - cache కి write, DB కి async later (fast కానీ data loss risk).

Singleflight - "thundering herd" (cache stampede) సమస్య పరిష్కారం. Popular key expire అయిన క్షణం, వెయ్యి concurrent requests ఒకేసారి DB కి వెళ్తాయి (అన్నీ miss). `golang.org/x/sync/singleflight` → **duplicate concurrent calls ని ఒక్క call గా collapse** చేస్తుంది - ఒక్కటే DB కి వెళ్తుంది, మిగతావి దాని result share చేసుకుంటాయి.

Real-life Scenario

Cache-aside = ఇంట్లో fridge. వంట చేయాలంటే ముందు fridge (cache) చూస్తావ్ - ఉంటే వాడతావ్ (hit). లేకపోతే market (DB) కి వెళ్లి తెచ్చి, fridge లో పెడతావ్ (next time కి). **Singleflight = ఇంట్లో పాలు అయిపోతే - నలుగురు ఒకేసారి "పాలు తేవాల్సి" అనుకుంటే, అందరూ విడివిడిగా market కి పరిగెత్తడం అనవసరం (thundering herd).** ఒక్కరు వెళ్లి అందరికీ తెస్తారు (single flight) - మిగతా ముగ్గురు ఇంట్లో wait చేసి పంచుకుంటారు.

Code — cache-aside + singleflight

```

package cache

import (
    "context"
    "encoding/json"
    "time"

    "github.com/redis/go-redis/v9"
    "golang.org/x/sync/singleflight"
)

type UserCache struct {
    rdb *redis.Client
    db   Store
    group singleflight.Group // duplicate concurrent loads collapse చేస్తుంది
}

// GetUser: cache-aside pattern + singleflight (thundering herd రక్షణ)
func (c *UserCache) GetUser(ctx context.Context, id string) (*User, error) {
    key := "user:" + id

    // 1. cache లో చూడు (hit అయితే వెంటనే return)
    if data, err := c.rdb.Get(ctx, key).Bytes(); err == nil {
        var u User
        json.Unmarshal(data, &u)
        return &u, nil // cache HIT
    }

    // 2. MISS → singleflight: ఒకే key కి concurrent loads ఒక్కటిగా collapse
    //   వెయ్యి goroutines ఇదే id అడిగినా, DB కి ఒక్కటే call వెళ్తుంది
    v, err, _ := c.group.Do(key, func() (any, error) {
        u, err := c.db.GetUser(ctx, id) // ఒక్కటే DB hit
        if err != nil {
            return nil, err
        }
        // 3. cache లో పెట్టు (TTL తో - stale కాకుండా)
        data, _ := json.Marshal(u)
        c.rdb.Set(ctx, key, data, 10*time.Minute) // TTL 10 min
        return u, nil
    })
    if err != nil {
        return nil, err
    }
    return v.(*User), nil
}

// invalidation: DB update అయితే cache delete (cache-aside stale ని fix)
func (c *UserCache) UpdateUser(ctx context.Context, u *User) error {
    if err := c.db.UpdateUser(ctx, u); err != nil {
        return err
    }
}

```

```
c.rdb.Del(ctx, "user:"+u.ID) // delete (next read fresh తెస్తుంది)
return nil
}
```

```
// ---- in-process LRU cache (fastest, per-instance) ----
import lru "github.com/hashicorp/golang-lru/v2"

cache, _ := lru.New[string, *User](10_000) // max 10K entries (LRU eviction)
cache.Add("user:1", &User{ID: "1"})
if u, ok := cache.Get("user:1"); ok {
    _ = u // hit - network లేదు, nanoseconds
}
```

Caching patterns — పోలిక

Pattern	Write flow	Read	Trade-off
Cache-aside	app DB కి; cache invalidate	app cache→DB miss	most common; miss latency, stale window
Write-through	app cache+DB (sync)	cache always fresh	write slow; cache DB mirror
Write-back	app cache; DB async	fast	data loss risk (cache crash)
Read-through	(cache lib DB చదువుతుంది)	app cache మాత్రమే	cache layer smart; less common in Go

In-process vs Redis

అంశం	In-process (LRU)	Redis (distributed)
Speed	nanoseconds (no network)	microseconds-ms (network)
Shared across instances	✗ (per-pod)	✓
Consistency	Pods diverge	shared, consistent
Size limit	process RAM	పెద్దది (dedicated)
Survives restart	✗	✓ (persistence optional)
Best fit	hot small data, config	shared state, sessions, big cache

Trade-offs

- **In-process:** fastest కానీ per-instance (10 pods = 10 copies, inconsistent), memory pressure, restart loss. Config/hot-tiny-data కి.
- **Redis:** shared + consistent + big కానీ network hop + Redis outage = cache down (fallback కావాలి) + extra infra.
- **TTL tuning:** short = fresh కానీ hit rate తక్కువ; long = fast కానీ stale. Business బట్టి.
- **Invalidation is hard:** "cache invalidation is one of the two hard problems." Cache-aside stale window ఉంటుంది - write మీద delete/update.

- **Two-tier:** in-process (L1) + Redis (L2) common - hot data in-process, rest Redis.

Key Points

- Read-heavy → cache. Cache-aside most common (miss → DB → populate).
- In-process (LRU) = fastest, per-instance. Redis = shared, consistent, distributed.
- **Singleflight** = thundering herd/stampede రక్షణ (duplicate concurrent loads collapse).
- ఎప్పుడూ **TTL** పెట్టు (unbounded cache = memory/stale risk). Write మీద invalidate.
- Redis down అయితే fallback (DB direct) - cache outage service ని కూల్చకూడదు.

Interview దృష్టి

"Popular product page cache expire అయిన వ్తణం DB కూలుతోంది - fix?" → "Cache stampede/thundering herd. singleflight తో duplicate concurrent DB loads ని ఒక్క call గా collapse చేస్తా - ఒక్కటే DB hit, మిగతావి result share. Plus jittered TTL (అన్ని keys ఒకేసారి expire కాకుండా), stale-while-revalidate (పాత value serve చేస్తూ background refresh)." singleflight చెప్పడం Go-specific senior signal.

12. Message queues in Go — Kafka, NATS, RabbitMQ

వివరణ

Message queue = producers events/messages ని queue కి రాస్తారు, consumers వాటిని process చేస్తారు - **async, decoupled**. Producer consumer వేగం తెలియనవసరం లేదు (buffer). HLD లో: spike absorption, decoupling, async work, event-driven.

మూడు popular options + Go clients:

- **Kafka** - distributed **log** (partitioned, replicated, persistent, replayable). High throughput, event streaming, ordered per-partition. Go: `segmentio/kafka-go` (pure Go, easy), `Shopify/sarama` (full-featured), `confluent-kafka-go` (librdkafka wrapper, fast కానీ CGO).
- **NATS** - lightweight pub/sub + JetStream (persistence). Very fast, simple, cloud-native (Go లో రాశారు). Microservices messaging కి great.
- **RabbitMQ** - traditional message **broker** (AMQP). Flexible routing (exchanges, queues), per-message ack. Task queues కి classic. Go: `amqp091-go`.

Delivery semantics - critical:

- **At-most-once** - deliver ఒకసారి లేదా అస్సలు కాదు (loss ok). Fast.
- **At-least-once** - deliver కనీసం ఒకసారి (duplicates వస్తాయి). **Idempotent consumer అవసరం**. Most common.
- **Exactly-once** - ఒక్కసారే (కష్టం, ఖరీదు, Kafka transactions పరిమితంగా). Practice లో "at-least-once + idempotency" = effectively-once.

Real-life Scenario

Kafka = restaurant order slips board. Waiter (producer) order slips ని board (log) మీద పెడతాడు - వరుసగా, ఎప్పటికీ ఉంటాయి (persistent, replayable). Chefs (consumers) వాటిని తీసుకుని వండుతారు. Waiter chef వేగం చూడనవసరం లేదు - board buffer చేస్తుంది (spike absorb). **At-least-once = chef ఒక slip రెండుసార్లు చూసినా** (network glitch) రెండు dishes వండకూడదు - "ఈ order ఇప్పటికే వందానా?" check (idempotency - order id తో). **RabbitMQ = chef ఒక్కో slip తీసుకుని "అయిపోయింది" అని board కి చెప్పడం (ack)** - Kafka లో board మీదే ఉంటాయి (offset), RabbitMQ లో తీసేస్తారు.

Code — Kafka producer + consumer (segmentio)

```

package main

import (
    "context"
    "log"
    "time"

    "github.com/segmentio/kafka-go"
)

// ---- Producer: events రాయడం ----
func produce(ctx context.Context) error {
    w := &kafka.Writer{
        Addr:      kafka.TCP("localhost:9092"),
        Topic:     "orders",
        Balancer:   &kafka.Hash{}, // same key → same partition (ordering per key)
        RequiredAcks: kafka.RequireAll, // అన్ని replicas ack (durability; RequireOne
= fast)
        BatchTimeout: 10 * time.Millisecond,
    }
    defer w.Close()

    // key ముఖ్యం: same order_id ఎప్పుడూ same partition → per-order ordering
    return w.WriteMessages(ctx, kafka.Message{
        Key:   []byte("order-123"),
        Value: []byte(`{"order_id":"123","status":"PAID"}`),
    })
}

// ---- Consumer: at-least-once + idempotency ----
func consume(ctx context.Context, processed IdempotencyStore) error {
    r := kafka.NewReader(kafka.ReaderConfig{
        Brokers: []string{"localhost:9092"},
        Topic:   "orders",
        GroupID: "order-processor", // consumer group → partitions పంచుకుంటారు
(scale)
        // manual commit → process అయ్యాకే offset commit (at-least-once)
        CommitInterval: 0,
    })
    defer r.Close()

    for {
        msg, err := r.FetchMessage(ctx) // fetch (commit కాదు)
        if err != nil {
            return err
        }
        // idempotency: duplicate message (retry) రెండేసారి process చేయకు
        msgID := string(msg.Key) + "-" + string(rune(msg.Offset))
        if seen, _ := processed.Exists(ctx, msgID); seen {
            r.CommitMessages(ctx, msg) // ఇప్పటికే process - skip + commit
            continue
        }
    }
}

```

```

    }

    if err := handleOrder(msg.Value); err != nil {
        log.Printf("process failed, NOT committing (will retry): %v", err)
        continue // commit చేయలేదు → మళ్ళీ deliver (at-least-once)
    }

    processed.Mark(ctx, msgID) // processed గా mark (dedup)
    r.CommitMessages(ctx, msg) // * process అయ్యాక commit (crash → replay)
}

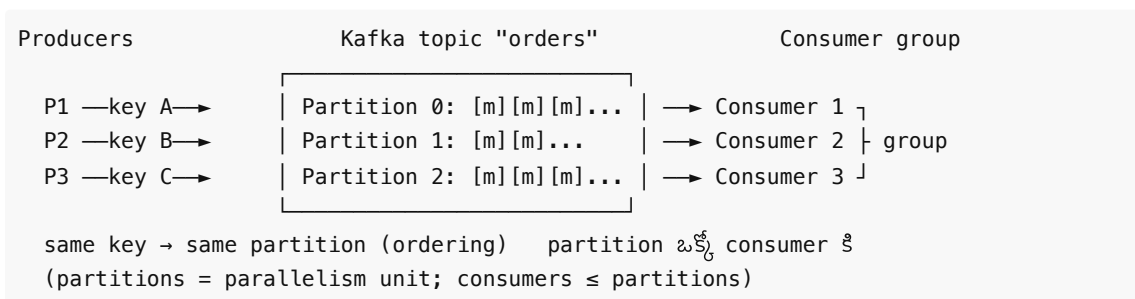
func handleOrder(data []byte) error { return nil }

```

Queue systems — పోలిక

అంశం	Kafka	NATS (JetStream)	RabbitMQ
Model	distributed log	pub/sub + stream	broker (AMQP)
Persistence	✅ (replayable, retention)	✅ (JetStream)	✅ (durable queues)
Throughput	చాలా ఎక్కువ	ఎక్కువ	మధ్యం
Ordering	per-partition	per-subject	per-queue
Replay	✅ (offsets)	✅	❌ (consumed = gone)
Routing flexibility	తక్కువ (topic/partition)	subjects (wildcards)	ఎక్కువ (exchanges)
Best fit	event streaming, high-throughput, replay	lightweight microservices msg	task queues, complex routing
Go client	segmentio/sarama	nats.go (native)	amqp091-go
Written in	Scala/Java	Go	Erlang

Producer/consumer scaling (diagram)



Trade-offs

- **At-least-once + idempotency:** practical default. Consumer duplicate-safe గా ఉండాలి (dedup store / idempotent op). Offset **process అయ్యాకే commit**.
- **Ordering vs parallelism:** per-partition ordering, కానీ parallelism = partition count. Global ordering కావాలంటే 1 partition (throughput పోతుంది).
- **Consumer lag:** consumers producers కంటే slow అయితే lag పెరుగుతుంది - monitor చేయి (Kafka lag metric). Scale consumers ($\leq$ partitions).
- **Kafka vs RabbitMQ:** Kafka = high-throughput streaming + replay; RabbitMQ = flexible routing + per-message ack + task queues. NATS = lightweight, fast, simple.

Key Points

- MQ = async + decoupling + spike absorption. Producer/consumer వేగం independent.
- **At-least-once + idempotent consumer** = practical default (duplicates handle చేయి).
- Offset ని **process అయ్యాకే commit** (crash $\rightarrow$ replay, loss కాదు).
- Kafka = log/streaming/replay; NATS = lightweight (Go native); RabbitMQ = routing/task queues.
- Partitions = parallelism + per-partition ordering. Consumers $\leq$ partitions.
- Consumer lag monitor చేయి.

Interview దృష్టి

"Order events process చేసేటప్పుడు duplicates - fix?" $\rightarrow$ "At-least-once delivery కాబట్టి duplicates natural. Idempotent consumer: message id/key తో dedup store (Redis) - already processed అయితే skip. Offset ని process + mark అయ్యాకే commit (crash-safe replay). Effectively-once = at-least-once + idempotency. Ordering కావాలంటే key తో same partition." Delivery semantics + idempotency clarity = senior signal.

Part 4 — Distributed Systems Building Blocks (Go lens)

13. Load balancing & service discovery (L4/L7, client-side LB, Consul/etcd)

వివరణ

Load balancer (LB) = incoming traffic ని multiple backend instances మీద పంచుతుంది $\rightarrow$ horizontal scale + availability (ఒక instance చస్తే traffic మిగతావాటికీ). రెండు layers:

- **L4 (transport)** - TCP/UDP level. IP+port చూసి route (packet contents చూడదు). Fast, protocol-agnostic. ఉదా AWS NLB.
- **L7 (application)** - HTTP level. URL path, headers, cookies చూసి route (/api $\rightarrow$ service A, /img $\rightarrow$ service B). Smart కానీ slower. ఉదా Nginx, Envoy, AWS ALB.

LB algorithms: round-robin, least-connections, weighted, IP-hash (sticky), consistent-hash.

Service discovery - services dynamic గా (autoscale, crash, redeploy) వస్తూ పోతూ ఉంటాయి. IP hardcode కుదరదు. Discovery = "service X ఇప్పుడు ఏ IPs మీద ఉంది" తెలుసుకోవడం. **Consul, etcd** (రెండూ Go!) - services register అవుతాయి, clients query చేస్తాయి. Kubernetes లో built-in (DNS + Services).

Client-side LB - proxy (LB box) లేకుండా, **client తనే** available instances list తెచ్చి (discovery నుంచి) దానిలో ఒకటి ఎంచుకుంటుంది. Extra network hop లేదు. **gRPC-Go దీన్ని native గా చేస్తుంది** - name resolver + balancer (round-robin, pick-first).

Real-life Scenario

L4 LB = postman house number చూసి deliver (packet ని తెరవడు, IP:port మాత్రమే). **L7 LB = receptionist letter తెరిచి "ఇది accounts కి, ఇది HR కి" అని content బట్టి route** (URL/header చూస్తుంది). **Service discovery = company phone directory** - ఉద్యోగులు వస్తూ పోతూ ఉంటారు (autoscale), directory (Consul/etcd) ఎప్పుడూ current extensions చెప్పుంది. **Client-side LB = నీ దగ్గరే directory copy** - operator (proxy) ద్వారా కాకుండా నువ్వే directly available extension కి call చేస్తావు (ఒక hop తగ్గింది).

Code — reverse proxy + client-side LB

```
// ---- L7 reverse proxy LB (stdlib httputil - Go లో సులభం!) ----
package main

import (
    "net/http"
    "net/http/httputil"
    "net/url"
    "sync/atomic"
)

type LoadBalancer struct {
    backends []*url.URL
    counter  atomic.Uint64 // round-robin counter (atomic - concurrent safe)
}

func (lb *LoadBalancer) next() *url.URL {
    // round-robin: counter++ % len (atomic → lock లేకుండా)
    i := lb.counter.Add(1) % uint64(len(lb.backends))
    return lb.backends[i]
}

func (lb *LoadBalancer) ServeHTTP(w http.ResponseWriter, r *http.Request) {
    target := lb.next()
    proxy := httputil.NewSingleHostReverseProxy(target) // stdlib reverse proxy
    proxy.ServeHTTP(w, r) // request ని backend కి forward
}

func main() {
    lb := &LoadBalancer{backends: []*url.URL{
        mustURL("http://10.0.0.1:8080"),
        mustURL("http://10.0.0.2:8080"),
        mustURL("http://10.0.0.3:8080"),
    }}
    http.ListenAndServe(":80", lb)
}

func mustURL(s string) *url.URL { u, _ := url.Parse(s); return u }
```

```
// ---- gRPC client-side LB (native - proxy లేదు) ----
import (
    "google.golang.org/grpc"
    "google.golang.org/grpc/balancer/roundrobin"
    _ "google.golang.org/grpc/health" // health-based picking
```

```

)

// dns:/// resolver → అన్ని A records తెచ్చి, round-robin గా పంచుతుంది
conn, err := grpc.NewClient(
    "dns:///order-service.default.svc.cluster.local:9090", // k8s headless service
    grpc.WithDefaultServiceConfig(
        `{"loadBalancingConfig":[{"round_robin":{}}]}`, // client-side round-robin
    grpc.WithTransportCredentials(insecure.NewCredentials()),
)
// client ప్రతి RPC ని వేరే backend కి పంపుతుంది – proxy hop లేదు
_ = err
_ = roundrobin.Name

```

L4 vs L7 — పోలిక

అంశం	L4 (TCP/UDP)	L7 (HTTP)
ఏ level	transport (IP:port)	application (URL, headers)
Speed	fast (no parse)	slower (parse HTTP)
Routing	connection-based	content-based (path, host, cookie)
TLS termination	❌ (passthrough)	✅ (decrypt, inspect)
Sticky sessions	IP-hash	cookie-based
ఉదా	AWS NLB, HAProxy(tcp)	Nginx, Envoy, ALB

Server-side vs Client-side LB

అంశం	Server-side (proxy)	Client-side
Where	dedicated LB box	client library
Extra hop	✅ (proxy)	❌ (direct)
Client complexity	simple	discovery + balancing logic
Latency	+1 hop	తక్కువ
ఉదా	Nginx, ALB	gRPC-Go, Finagle, service mesh sidecar

Trade-offs

- **L4 vs L7:** L4 fast + simple; L7 smart routing (path-based, canary) కానీ CPU cost + TLS termination. Modern setups Envoy (L7) + NLB (L4) కలిపి.
- **Client-side LB:** hop తగ్గుతుంది + smart (health-aware) కానీ client fat (every language కి logic) → service mesh (Istio/Envoy sidecar) దీన్ని offload చేస్తుంది.
- **Discovery consistency:** stale entries (crashed instance ఇంకా list లో) → health checks + TTL అవసరం.

Key Points

- LB = scale + availability. L4 (IP:port, fast) vs L7 (HTTP content, smart).

- Service discovery = dynamic instance IPs (Consul/etcd, k8s DNS). Hardcode కాదు.
- **gRPC-Go client-side LB native** - resolver + round-robin, proxy hop లేదు.
- stdlib httputil.ReverseProxy తో Go లో L7 LB few lines.
- Health checks + TTL → stale/dead instances remove.

Interview దృష్టి

"Go microservices ఒకదానితో ఒకటి ఎలా discover/balance?" → "Kubernetes లో: service DNS + kube-proxy (L4) లేదా gRPC client-side LB (dns:/// resolver + round_robin - proxy hop తప్పుతుంది, health-aware). Larger: service mesh (Envoy sidecar) - LB, mTLS, retries offload. Consul/etcd non-k8s discovery కీ." Client-side LB + mesh distinction senior signal.

14. SQL vs NoSQL, replication, sharding/partitioning

వివరణ

SQL (relational) - fixed schema, ACID transactions, joins, strong consistency. Postgres, MySQL. Relations + consistency ముఖ్యమైనప్పుడు (money, orders).

NoSQL - flexible schema, horizontal scale, high throughput. Types: document (MongoDB), key-value (DynamoDB, Redis), wide-column (Cassandra), graph (Neo4j). Scale + flexible data + simple access patterns కీ.

Replication = same data ని multiple nodes లో copy → read scale + availability + DR.

- **Leader-follower (primary-replica)** - writes leader కీ, reads followers నుంచి. **Replication lag** (follower వెనకబడింపు → stale reads). Failover: leader చస్తే follower promote.
- **Multi-leader** - multiple write nodes (conflict resolution అవసరం).
- **Sync vs async replication** - sync (leader follower ack కోసం wait - safe కానీ slow) vs async (fast కానీ leader crash → data loss).

Sharding/Partitioning = data ని multiple DBs మీద **split** (replication = copy; sharding = split). ఒక్క DB capacity దాటినప్పుడు.

- **Hash sharding** - `hash(key) % N` → even distribution కానీ range queries కష్టం, resharding painful (consistent hashing పరిష్కారం).
- **Range sharding** - key ranges (A-M, N-Z) → range queries easy కానీ hotspots.
- **Directory/lookup** - separate service maps key → shard.

Real-life Scenario

Replication = ఒకే పుస్తకం Xerox copies - అందరూ ఒకేసారి చదవొచ్చు (read scale), ఒక copy చిరిగినా మిగతావి ఉన్నాయి (availability). కానీ master copy update చేస్తే copies కీ propagate అవ్వడానికి కొంత సమయం (replication lag → stale read). **Sharding = library ని branches ని విడగొట్టడం** - ఒక branch లో A-M పుస్తకాలు, ఇంకో branch లో N-Z. ఒక్క building లో అన్నీ వట్టివు (capacity) కాబట్టి split. కానీ "A నుంచి Z దాకా అన్ని books" (cross-shard query) కావాలంటే అన్ని branches కీ వెళ్ళాలి (కష్టం).

Code — Go లో read/write splitting + sharding

```
// ---- Read/write splitting (leader writes, replica reads) ----
type DB struct {
    leader *sql.DB // writes (primary)
    replicas []*sql.DB // reads (followers) - round-robin
```

```

    rr      atomic.Uint64
}

// Write → ఎప్పుడూ leader (consistency)
func (d *DB) Exec(ctx context.Context, q string, args ...any) error {
    _, err := d.leader.ExecContext(ctx, q, args...)
    return err
}

// Read → replica (scale). కానీ replication lag → stale కావచ్చు!
func (d *DB) Query(ctx context.Context, q string, args ...any) (*sql.Rows, error) {
    i := d.rr.Add(1) % uint64(len(d.replicas))
    return d.replicas[i].QueryContext(ctx, q, args...)
}

// read-your-writes కావాలంటే → leader నుంచి చదువు (freshly written data)
func (d *DB) QueryStrong(ctx context.Context, q string, args ...any) (*sql.Rows, error) {
    return d.leader.QueryContext(ctx, q, args...) // fresh (no lag)
}

```

```

// ---- Hash sharding: key → ఏ shard ----
type ShardedStore struct {
    shards []*sql.DB // ఒక్కో shard వేరే DB server
}

func (s *ShardedStore) shardFor(key string) *sql.DB {
    h := fnv.New32a()
    h.Write([]byte(key))
    return s.shards[h.Sum32()%uint32(len(s.shards))] // hash % N
}

func (s *ShardedStore) GetUser(ctx context.Context, userID string) (*User, error) {
    db := s.shardFor(userID) // ఆ user ఏ shard లో
    var u User
    err := db.QueryRowContext(ctx,
        `SELECT id, email FROM users WHERE id=$1`, userID).Scan(&u.ID, &u.Email)
    return &u, err
    // cross-shard query (అన్ని users) → అన్ని shards query చేసి merge చేయాలి (కష్టం)
}

```

SQL vs NoSQL — పోలిక

అంశం	SQL	NoSQL
Schema	fixed (rigid)	flexible
Transactions	ACID (strong)	limited (కొన్ని)
Joins	✓	✗ (denormalize)

Scale	vertical (harder horizontal)	horizontal (built-in)
Consistency	strong	eventual (సాధారణంగా)
Best fit	relations, money, complex queries	scale, flexible, simple access
ఉదా	Postgres, MySQL	Mongo, Cassandra, DynamoDB

Sharding strategies

Strategy	ఎలా	✓	✗
Hash	<code>hash(key)%N</code>	even distribution	range query కష్టం, reshard painful
Range	A-M, N-Z	range query easy	hotspots (uneven)
Consistent hash	ring	reshard minimal	complexity
Directory	lookup service	flexible	lookup = extra hop, SPOF

Trade-offs

- **SQL first:** relations + transactions ఉంటే SQL. "NoSQL = scale" గుడ్డిగా వాడకు - access pattern బట్టి. చాలా apps కి Postgres single node చాలా దూరం వెళ్తుంది (millions users).
- **Replication lag:** async replicas fast + scale కానీ stale reads. Read-your-writes కావాలంటే leader నుంచి / sticky.
- **Sharding = last resort:** cross-shard joins/transactions కష్టం, resharding painful, operational complexity. ముందు: vertical scale, read replicas, caching. అవి చాలకపోతేనే shard.
- **Choose shard key carefully:** bad key → hotspots. High-cardinality, even-access key.

Key Points

- SQL = relations/ACID; NoSQL = scale/flexible. Access pattern బట్టి, hype బట్టి కాదు.
- Replication = copy (read scale + HA); Sharding = split (write/capacity scale).
- Replica reads fast కానీ **replication lag → stale**. Read-your-writes → leader.
- Sharding = last resort (cross-shard pain). Shard key carefully (hotspots avoid).
- Go: read/write splitting = leader/replica pools; sharding = `hash(key)%N` → pool select.

Interview దృష్టి

"10M users - shard చేస్తావా?" → "వెంటనే కాదు. ముందు: Postgres primary + read replicas (read scale) + caching (Redis) + proper indexes + connection pooler. Single primary millions users handle చేస్తుంది. Write throughput/data size ఒక్క node capacity దాటినప్పుడే shard - high-cardinality even-access shard key (user_id), cross-shard queries minimize. Sharding operational cost ఎక్కువ." "Shard as last resort" = maturity.

15. Consistency, CAP/PACELC, consensus (Raft) — etcd/hashicorp-raft

వివరణ

CAP theorem - distributed system లో network **partition (P)** వచ్చినప్పుడు (nodes మధ్య communication తెగింది), **Consistency (C)** లేదా **Availability (A)** - ఒక్కటి ఎంచుకోగలవు, రెండూ కాదు.

- **CP** - partition లో consistency కాపాడి, availability వదులు (కొన్ని nodes requests reject). ఉదా: etcd, ZooKeeper, HBase. (Money, config, locks).
- **AP** - partition లో available గా ఉండి, consistency వదులు (stale data serve). ఉదా: Cassandra, DynamoDB. (Social feed, cart).

PACELC - CAP extension: partition (P) లో A vs C; **Else (E)** - normal ఆపరేషన్ లో **Latency (L)** vs **Consistency (C)**. అంటే partition లేకపోయినా consistency కి latency price. ఉదా: DynamoDB = PA/EL (available + low latency), Spanner = PC/EC (consistent always).

Consistency models: strong (linearizable - real-time order), sequential, causal, eventual (converge eventually).

Consensus (Raft) - multiple nodes ఒక value మీద **agree** అవ్వడం (leader election, replicated log). Raft = understandable consensus (Paxos alternative). **etcd, Consul, CockroachDB, TiKV** అన్నీ Raft. Go లో `hashicorp/raft`, `etcd/raft` libraries.

Raft basics: nodes elect ఒక **leader**; leader అన్ని writes తీసుకుని **log** గా followers కి replicate; **majority (quorum)** ack అయితే commit. Leader చస్తే election → కొత్త leader. Split-brain avoid (majority అవసరం).

Real-life Scenario

CAP = ఇద్దరు cashiers, phone line తెగింది. ఒకే bank account కి రెండు counters. Phone (network) పనిచేస్తున్నప్పుడు ఇద్దరూ balance sync చేసుకుంటారు. Phone తెగితే (partition) - ఒక option: "line వచ్చేదాకా withdrawal అపు" (CP - consistent కానీ unavailable). ఇంకో option: "ఇద్దరూ విడిగా withdrawal ఇవ్వండి, తర్వాత sync" (AP - available కానీ overdraw risk = inconsistent). **Raft = committee majority vote** - ఏ నిర్ణయమైనా majority (quorum) ఒప్పుకోవాలి. Leader (chairman) చస్తే, మిగతా members కొత్త chairman ని elect చేస్తారు - majority ఉంటేనే valid (split-brain లో రెండు chairmen అవ్వకుండా).

Code — etcd (Raft-backed) leader election + Go client

```
// ---- etcd: distributed config + leader election (Raft internally) ----
package main

import (
    "context"
    "log"
    "time"

    clientv3 "go.etcd.io/etcd/client/v3"
    "go.etcd.io/etcd/client/v3/concurrency"
)

func leaderElection() error {
    cli, err := clientv3.New(clientv3.Config{
        Endpoints:    []string{"localhost:2379"},
        DialTimeout: 5 * time.Second,
    })
    if err != nil {
        return err
    }
    defer cli.Close()

    // session: lease (TTL) - process చస్తే leadership auto-release
```

```

session, err := concurrency.NewSession(cli, concurrency.WithTTL(10))
if err != nil {
    return err
}
defer session.Close()

election := concurrency.NewElection(session, "/leader/order-service")
ctx := context.Background()

// Campaign: leader అయ్యేదాకా block (majority quorum ద్వారా Raft ఎంచుకుంటుంది)
log.Println("leadership కోసం campaign...")
if err := election.Campaign(ctx, "node-1"); err != nil {
    return err
}
log.Println("నేను leader! (singleton work చేయొచ్చు - cron, coordination)")
// leader మాత్రమే చేయాల్సిన పని ఇక్కడ (single writer, scheduler)
// ...
return election.Resign(ctx) // వదులుకో (graceful)
}

// ---- strongly-consistent read/write (etcd = CP - linearizable) ----
func kvOps(cli *clientv3.Client) {
    ctx := context.Background()
    cli.Put(ctx, "config/feature_x", "enabled") // majority commit అయ్యాకే success
    resp, _ := cli.Get(ctx, "config/feature_x") // linearizable read (fresh)
    _ = resp
}

```

CAP choices — పోలిక

System	CAP	ఎప్పుడు
etcd, ZooKeeper	CP	config, locks, leader election, service discovery
Spanner, CockroachDB	CP (Cts sync)	global consistent SQL
Cassandra, DynamoDB	AP	high availability, scale, eventual ok
Postgres (single)	CA*	single node (partition లేదు)

* single-node "CA" theoretical - real distributed లోP తప్పదు.

Consistency models

Model	అర్థం	ఉదా
Strong (linearizable)	real-time single order, ఎప్పుడూ fresh	etcd, Spanner
Sequential	some consistent order (real-time కాదు)	-
Causal	cause-effect preserved	collaborative apps
Eventual	eventually converge	DNS, Cassandra, S3

Trade-offs

- **CP vs AP business-driven:** money/locks/config → CP (etcd). Social feed/cart/likes → AP (eventual ok, availability ముఖ్యం).
- **Strong consistency = latency + lower availability:** consensus (majority ack) రౌండ్ ట్రిప్లు. అందుకే PACELC "EL vs EC" - normal లో కూడా consistency కి latency.
- **Raft quorum:** N nodes → (N/2)+1 majority అవసరం. 3 nodes → 2, 5 nodes → 3. Odd numbers (waste తక్కువ). Majority లేకపోతే writes ఆగుతాయి (CP).
- **Don't roll your own consensus:** etcd/Consul/hashicorp-raft వాడు. Consensus bugs subtle (split-brain, data loss).

Key Points

- CAP: partition లో **C లేదా A** (P తప్పదు distributed లో). PACELC: else లో L vs C కూడా.
- CP (etcd/ZK) = config/locks/leader; AP (Cassandra/Dynamo) = scale/availability.
- **Raft** = consensus (leader election + replicated log + majority commit). etcd/Consul/CockroachDB.
- Quorum = majority ((N/2)+1). Split-brain avoid. Odd node counts.
- Consensus roll your own చేయకు - etcd/hashicorp-raft.
- Leader election Go లో: etcd concurrency.Election (singleton work - cron, coordination).

Interview దృష్టి

"Distributed system లో ఒక్క node మాత్రమే cron job run చేయాలి - ఎలా?" → "Leader election. etcd/Consul (Raft-backed) Election - lease TTL తో ఒక node leader అవుతుంది, అది మాత్రమే job run చేస్తుంది. Leader చస్తే lease expire → కొత్త election (automatic failover). Roll-my-own లాక్ కాదు - split-brain/fencing subtle. CP system ఎందుకంటే coordination correctness ముఖ్యం." Raft/etcd + fencing awareness = strong senior signal.

16. Idempotency, distributed locks, distributed transactions (Saga)

వివరణ

Idempotency (Topic 7 లో API context) - operation ఎన్నిసార్లు చేసినా result ఒక్కటే. Retries + at-least-once delivery ఉన్న distributed world లో essential.

Distributed locks - multiple instances లో ఒక్క resource ని ఒకేసారి ఒక్కరే access చేయాలి (mutual exclusion across machines). Redis (SET NX + TTL), etcd (lease). **జాగ్రత్త:** distributed lock hard - TTL expire అయితే lock holder ఇంకా పని చేస్తుండగా వేరే వాడికి lock వస్తుంది → **fencing token** (monotonic id) అవసరం.

Distributed transactions - multiple services/DBs మీద atomic operation. Traditional **2PC (two-phase commit)** - coordinator అందరినీ "ready?" అడిగి, అందరూ yes అంటే commit. కానీ blocking + coordinator SPOF → microservices లో avoid.

Saga - distributed transaction ని **sequence of local transactions** గా, ప్రతిదానికి **compensating action** (undo). Fail అయితే already-done steps ని compensate (rollback). రెండు రకాలు: **choreography** (events, decentralized) vs **orchestration** (central coordinator).

Real-life Scenario

Distributed lock = single toilet key (office లో ఒకటి). Key ఉన్నవాడే వాడగలడు (mutual exclusion). కానీ key తీసుకున్నవాడు మర్చిపోతే (crash) - timeout తర్వాత spare key ఇస్తారు (TTL). సమస్య: timeout అయ్యాక పాత వాడు ఇంకా లోపల ఉంటే, కొత్త వాడు కూడా వెళ్ళాడు (రెండు writers)! అందుకే key కి number (fencing token) - resource "ఎక్కువ number key చూపించినవాడిదే valid" అంటుంది. **Saga = online order = payment**

+ **inventory + shipping**. Shipping fail అయితే - already-charged payment ని **refund** (compensate), reserved inventory **release**. ఒక్కో step కి "undo" - forward fail అయితే వెనక్కి unwind.

Code — Redis distributed lock (fencing) + Saga

```
// ---- Redis distributed lock with fencing token ----
package lock

import (
    "context"
    "errors"
    "time"

    "github.com/redis/go-redis/v9"
)

type DistLock struct {
    rdb *redis.Client
}

// Acquire: SET key value NX EX ttl → atomic (ఒక్కరే పొందగలరు)
// value = unique token (release లో ownership verify + fencing)
func (l *DistLock) Acquire(ctx context.Context, key, token string, ttl
time.Duration) (bool, error) {
    // NX = key లేకపోతేనే set (atomic mutual exclusion); EX = auto-expire (deadlock
    avoid)
    ok, err := l.rdb.SetNX(ctx, key, token, ttl).Result()
    return ok, err
}

// Release: నా token అయితేనే delete (వేరేవాడి lock ని పొరపాటున delete చేయకూడదు)
// Lua script → check-and-delete atomic
var releaseScript = redis.NewScript(`
    if redis.call("GET", KEYS[1]) == ARGV[1] then
        return redis.call("DEL", KEYS[1])
    end
    return 0`)

func (l *DistLock) Release(ctx context.Context, key, token string) error {
    return releaseScript.Run(ctx, l.rdb, []string{key}, token).Err()
}

var ErrLockHeld = errors.New("lock held by another")
```

Fencing: TTL expire అయితే lock holder ఇంకా పని చేస్తుండగా వేరేవాడు lock పొందొచ్చు. పరిష్కారం: monotonically increasing **fencing token** (etcd revision / Redis INCR) - resource "highest token మాత్రమే accept" చేస్తుంది. Redis lock alone perfectly safe కాదు (Redlock controversial) - critical correctness కి etcd/ZooKeeper.

```
// ---- Saga (orchestration): steps + compensations ----
type SagaStep struct {
```

```

    Name      string
    Action    func(ctx context.Context) error // forward
    Compensate func(ctx context.Context) error // undo
}

func RunSaga(ctx context.Context, steps []SagaStep) error {
    var completed []SagaStep
    for _, step := range steps {
        if err := step.Action(ctx); err != nil {
            // fail → already-completed steps & reverse order & compensate
            for i := len(completed) - 1; i >= 0; i-- {
                if cerr := completed[i].Compensate(ctx); cerr != nil {
                    // compensation fail = manual intervention (log + alert)
                    log.Printf("CRITICAL: compensation failed: %s: %v",
                        completed[i].Name, cerr)
                }
            }
            return fmt.Errorf("saga failed at %s: %w", step.Name, err)
        }
        completed = append(completed, step)
    }
    return nil // అన్ని steps success
}

// usage: order saga
steps := []SagaStep{
    {Name: "payment",
        Action: func(ctx context.Context) error { return charge(ctx) },
        Compensate: func(ctx context.Context) error { return refund(ctx) }},
    {Name: "inventory",
        Action: func(ctx context.Context) error { return reserve(ctx) },
        Compensate: func(ctx context.Context) error { return release(ctx) }},
    {Name: "shipping",
        Action: func(ctx context.Context) error { return ship(ctx) },
        Compensate: func(ctx context.Context) error { return cancelShip(ctx) }},
}

```

2PC vs Saga — పోలిక

అంశం	2PC	Saga
Atomicity	strong (all-or-nothing)	eventual (compensations)
Blocking	✅ (locks held)	❌ (non-blocking)
Coordinator SPOF	✅	orchestrator (or none)
Isolation	✅	❌ (intermediate states visible)
Scale	poor	good
Best fit	single DB, tight coupling	microservices

Saga: choreography vs orchestration

అంశం	Choreography	Orchestration
Control	decentralized (events)	central orchestrator
Coupling	loose	orchestrator knows all
Visibility	hard (flow scattered)	easy (one place)
Best fit	simple flows	complex flows

Trade-offs

- **Distributed lock risk:** TTL too short → lock expires mid-work (safety violated); too long → crash అయితే others long wait. Fencing token అవసరం. Redis Redlock debated - correctness-critical కి etcd/ZK.
- **Saga no isolation:** intermediate states visible (order "processing" లో inventory reserved కానీ payment ఇంకా కాదు). Business ఇది handle చేయాలి (pending states).
- **Compensation may fail:** refund fail అయితే? → retry + dead-letter + manual. Compensations కూడా idempotent + retryable ఉండాలి.
- **Prefer avoiding distributed transactions:** aggregate boundaries సరిగ్గా design చేస్తే single-service transaction సరిపోతుంది (DDD). Saga = అవసరమైతేనే.

Key Points

- Idempotency = retry/at-least-once world లో essential (dedup keys).
- Distributed lock: Redis SET NX EX + token release (Lua). **Fencing token** critical (TTL expiry race). Correctness → etcd/ZK.
- 2PC = blocking + SPOF → microservices లో avoid. **Saga** = local txns + compensations (non-blocking, eventual).
- Saga orchestration (central, visible) vs choreography (events, loose). Compensations idempotent + retryable.
- Distributed transactions ని avoid చేయగలిగితే avoid (aggregate boundaries).

Interview దృష్టి

"Order = payment + inventory + shipping మూడు services - atomic ఎలా?" → "2PC కాదు (blocking, SPOF). Saga - ఒక్కో local transaction + compensating action. Orchestrator steps నడిపి, fail అయితే reverse order లో compensate (refund, release). Each step + compensation idempotent (retries), no isolation కాబట్టి pending states business handle చేస్తుంది. Distributed lock అవసరమైతే fencing token తో." Saga + compensation + fencing = senior distributed systems depth.

Part 5 — Resilience & Concurrency at Scale

17. Concurrency at scale — worker pools, bounded concurrency, errgroup, fan-out/fan-in

వివరణ

Go strength = concurrency. కానీ "unbounded goroutines" = ప్రమాదం. ప్రతి request కి go doWork() అని పరిమితి లేకుండా goroutines spawn చేస్తే - million requests → million goroutines → memory blow, downstream (DB) overwhelm, crash. Senior Go = **bounded, controlled concurrency**.

- **Worker pool** - fixed సంఖ్య goroutines (workers) ఒక channel నుంచి jobs తీసుకుని process.
Concurrency capped (ఉదా 50 workers), backpressure natural (channel full → producer waits).
- **Bounded concurrency (semaphore)** - N concurrent operations మాత్రమే.
golang.org/x/sync/semaphore లేదా buffered channel (chan struct{} size N).
- **Backpressure** - downstream slow అయితే upstream ని slow చేయడం (queue full → reject/wait).
లేకపోతే unbounded buffering → OOM.
- **errgroup** - golang.org/x/sync/errgroup - multiple goroutines run చేసి, first error వస్తే మిగతావి cancel (context), అన్నీ wait. Concurrent sub-tasks + error handling clean.
- **Fan-out/fan-in** - ఒక్క request ని multiple parallel sub-tasks గా (fan-out), results కలపడం (fan-in).
ఉదా: product page = product + reviews + recommendations (parallel).

Real-life Scenario

Worker pool = టికెట్ counter. అపరిమిత counters తెరవలేవు (అపరిమిత goroutines) - hall లో జాగా, staff (memory, DB connections) పరిమితం. అందుకే fixed 10 counters (workers). Customers (jobs) queue (channel) లో వస్తారు, free counter వాళ్ళను serve చేస్తుంది. Queue నిండితే కొత్తవాళ్ళు బయట wait (backpressure) - hall crush అవుతుంది. **Fan-out/fan-in = వంట** - ఒక meal కి కూర, పప్పు, అన్నం ఒకేసారి వేర్వేరు stoves మీద వండి (fan-out), చివర ఒకే plate లో పెట్టడం (fan-in). Sequential గా చేస్తే 3x సమయం.

Code — worker pool + semaphore + errgroup fan-out

```
// ---- 1. Worker pool (bounded, backpressure) ----
func WorkerPool(ctx context.Context, jobs <-chan Job, workers int) <-chan Result {
    results := make(chan Result)
    var wg sync.WaitGroup

    // fixed సంఖ్య workers (ఇదే concurrency cap)
    for i := 0; i < workers; i++ {
        wg.Add(1)
        go func(id int) {
            defer wg.Done()
            for job := range jobs { // channel close అయ్యేదాకా jobs తీసుకో
                select {
                    case <-ctx.Done():
                        return // cancel → వెంటనే ఆగు
                    case results <- process(job):
                }
            }
        }(i)
    }
    // అన్ని workers అయ్యాక results close (fan-in cleanup)
    go func() { wg.Wait(); close(results) }()
    return results
}

// ---- 2. Bounded concurrency via semaphore (N concurrent max) ----
import "golang.org/x/sync/semaphore"
```

```

func ProcessAll(ctx context.Context, items []Item, maxConcurrent int64) error {
    sem := semaphore.NewWeighted(maxConcurrent) // N slots
    var wg sync.WaitGroup
    for _, item := range items {
        // Acquire: N అప్పటికే busy అయితే block (backpressure - unbounded spawn అవు)
        if err := sem.Acquire(ctx, 1); err != nil {
            break // context cancelled
        }
        wg.Add(1)
        go func(it Item) {
            defer wg.Done()
            defer sem.Release(1) // slot free
            handle(ctx, it)
        }(item)
    }
    wg.Wait()
    return ctx.Err()
}

// ---- 3. errgroup: fan-out sub-tasks, first error cancels all ----
import "golang.org/x/sync/errgroup"

type PageData struct {
    Product *Product
    Reviews []Review
    Recs    []Rec
}

// ProductPage: 3 independent calls PARALLEL (fan-out) → merge (fan-in)
func ProductPage(ctx context.Context, id string) (*PageData, error) {
    g, ctx := errgroup.WithContext(ctx) // ఏదైనా fail → ctx cancel → మిగతావి ఆగుతాయి
    var data PageData

    g.Go(func() error {
        p, err := fetchProduct(ctx, id)
        data.Product = p
        return err
    })
    g.Go(func() error {
        r, err := fetchReviews(ctx, id)
        data.Reviews = r
        return err
    })
    g.Go(func() error {
        rec, err := fetchRecs(ctx, id)
        data.Recs = rec
        return err
    })

    if err := g.Wait(); err != nil { // అన్నీ అయ్యేదాకా wait; first error
        return nil, err
    }
}

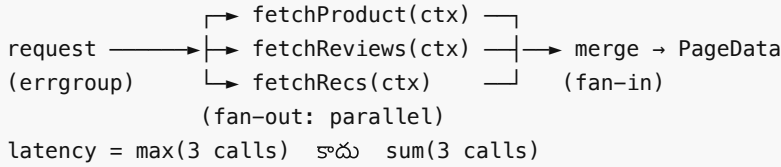
```

```

return &data, nil
// sequential అయితే 3 round-trips; parallel అయితే slowest ఒక్కటే (latency 1/3)
}

```

Fan-out/fan-in (diagram)



Concurrency patterns — పోలిక

Pattern	ఎప్పుడు	Go tool
Worker pool	stream of jobs, bounded workers	channel + N goroutines
Semaphore	N concurrent max (bounded)	x/sync/semaphore, buffered chan
errgroup	parallel sub-tasks + error/cancel	x/sync/errgroup
Fan-out/fan-in	one request → parallel sub-calls	errgroup / channels
Pipeline	multi-stage processing	chained channels

Trade-offs

- **Unbounded goroutines = disaster:** always cap (worker pool / semaphore). `go f()` per-request unbounded → OOM + downstream overwhelm.
- **Pool size tuning:** too small = underutilized; too large = downstream (DB) overwhelm + memory. Match downstream capacity (DB pool size తో align).
- **errgroup cancels on first error:** ఒక sub-task fail → మిగతావి cancel. Best-effort (partial results ok) కావాలంటే errgroup కాదు (errors collect చేయి).
- **Backpressure vs dropping:** channel full → block (backpressure, latency) vs drop (load shed, Topic 19). Business బట్టి.

Key Points

- **Never unbounded goroutines.** Worker pool / semaphore తో cap. Concurrency = capacity, infinite కాదు.
- Worker pool: fixed goroutines + job channel → bounded + natural backpressure.
- **errgroup** = parallel sub-tasks, first error cancels rest, `ctx` propagation. Fan-out/fan-in latency = slowest call, not sum.
- Pool size $\approx$ downstream capacity (DB pool align). Semaphore for bounded concurrent ops.
- ఎప్పుడూ `ctx` `watch (<-ctx.Done())` - cancel అయితే goroutines ఆగాలి (leak avoid).

Interview దృష్టి

"Product page 5 downstream calls చేస్తుంది - latency తగ్గించు." → "Independent calls parallel - errgroup తో fan-out/fan-in. Latency sum(5) కాకుండా max(5) అవుతుంది. Shared context - ఒకటి fail/timeout అయితే మిగతావి cancel (wasted work అవు). Downstream overwhelm కాకుండా bounded (semaphore/pool). Dependent calls మాత్రం sequential." Bounded + errgroup + context = Go concurrency maturity.

18. Resilience patterns — timeouts, retries, circuit breaker, bulkhead, rate limiting

వివరణ

Distributed systems లో failures **normal** (network, slow deps, overload). Resilient service = failures ని gracefully handle. Go లో core patterns:

- **Timeouts (context)** - ప్రతి downstream call కి deadline. `context.WithTimeout` . Timeout లేకపోతే ఒక slow dependency అన్ని goroutines block చేసి cascade → total outage. **అత్యంత ముఖ్యమైన resilience pattern.**
- **Retries with backoff** - transient failure అయితే retry, కానీ **exponential backoff + jitter** (వెంటనే కాదు - overwhelmed service ని ఇంకా hammer చేయకుండా). **Idempotent operations కి మాత్రమే.** Retry budget/cap.
- **Circuit breaker** - downstream consistently fail అయితే, calls ఆపిసి fast-fail (fuse లా). Downstream కి recover time ఇస్తుంది, caller resources waste చేయదు. States: **closed** (normal) → **open** (fail fast) → **half-open** (test). `sony/gobreaker` .
- **Bulkhead** - resources ని isolate (ఒక feature కి pool A, ఇంకో దానికి pool B) → ఒక feature overload అయినా మిగతావి బతుకుతాయి (ship's watertight compartments).
- **Rate limiting** - requests/sec cap (overload protection). Token bucket - `golang.org/x/time/rate` .

Real-life Scenario

Circuit breaker = ఇంటి fuse. Short circuit (downstream repeatedly failing) అయితే fuse కాలిపోతుంది (open) - మొత్తం wiring కాలకుండా current ఆపుతుంది. కొద్దిసేపటికి "సరేందా?" అని test చేస్తారు (half-open). సరేలే reset (closed). ప్రతినారి current పంపి (retry) ఇల్లు కాలబుకోవడం (cascade failure) కాదు. **Bulkhead = పడవలో watertight compartments** - ఒక భాగం లీక్ అయినా (ఒక feature crash) మొత్తం పడవ మునగదు (whole service down కాదు). **Retry with backoff = తలుపు కొట్టడం** - ఎవరూ తియ్యకపోతే వెంటనే మళ్ళీమళ్ళీ (hammering) కాదు - కొంచెం ఆగి, ఇంకొంచెం ఆగి (exponential).

Code — timeout + retry/backoff + circuit breaker

```
// ---- 1. Timeout (context) - ప్రతి downstream call కి ----
func fetchWithTimeout(ctx context.Context, url string) (*http.Response, error) {
    ctx, cancel := context.WithTimeout(ctx, 2*time.Second) // 2s deadline
    defer cancel()
    req, _ := http.NewRequestWithContext(ctx, "GET", url, nil)
    return http.DefaultClient.Do(req) // 2s దాటితే auto-cancel
}

// ---- 2. Retry with exponential backoff + jitter (idempotent only) ----
func retryWithBackoff(ctx context.Context, maxAttempts int, op func() error) error {
    var err error
    base := 100 * time.Millisecond
    for attempt := 0; attempt < maxAttempts; attempt++ {
        if err = op(); err == nil {
            return nil // success
        }
        if !isRetryable(err) {
            return err // permanent error (4xx) → retry అనవసరం
        }
    }
}
```

```

    }
    // exponential: 100ms, 200ms, 400ms... + jitter (thundering herd avoid)
    backoff := base * time.Duration(1<<attempt)
    jitter := time.Duration(rand.Int63n(int64(backoff / 2)))
    select {
    case <-ctx.Done():
        return ctx.Err() // overall deadline ಡಾಟಿ೦ದಿ
    case <-time.After(backoff + jitter):
    }
}
return fmt.Errorf("max retries exhausted: %w", err)
}

```

// ---- 3. Circuit breaker (sony/gobreaker) ----

```

import "github.com/sony/gobreaker"

func newBreaker() *gobreaker.CircuitBreaker {
    return gobreaker.NewCircuitBreaker(gobreaker.Settings{
        Name:      "payment-service",
        MaxRequests: 3, // half-open ಲ್ಲ test requests
        Interval:   10 * time.Second,
        Timeout:    30 * time.Second, // open → half-open ಸಿ wait
        ReadyToTrip: func(counts gobreaker.Counts) bool {
            // 5+ requests ಲ್ಲ 60%+ fail → open (fast-fail)
            ratio := float64(counts.TotalFailures) / float64(counts.Requests)
            return counts.Requests >= 5 && ratio >= 0.6
        },
    })
}

func callPayment(cb *gobreaker.CircuitBreaker, ctx context.Context) (any, error) {
    // open ಅಯಿತ್ಲೆ ವೆ೦ಟುನೆ error (downstream ಸಿ request ಫ೦ಪದು - fast fail)
    return cb.Execute(func() (any, error) {
        return fetchWithTimeout(ctx, "http://payment/charge")
    })
}

```

// ---- 4. Rate limiting (token bucket - golang.org/x/time/rate) ----

```

import "golang.org/x/time/rate"

// 100 req/sec sustained, burst 20
var limiter = rate.NewLimiter(rate.Limit(100), 20)

func RateLimit(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        if !limiter.Allow() { // token ಲೆಕ್ಕಪತ್ಲೆ reject
            http.Error(w, "rate limit exceeded", http.StatusTooManyRequests) // 429
            return
        }
        next.ServeHTTP(w, r)
    })
}

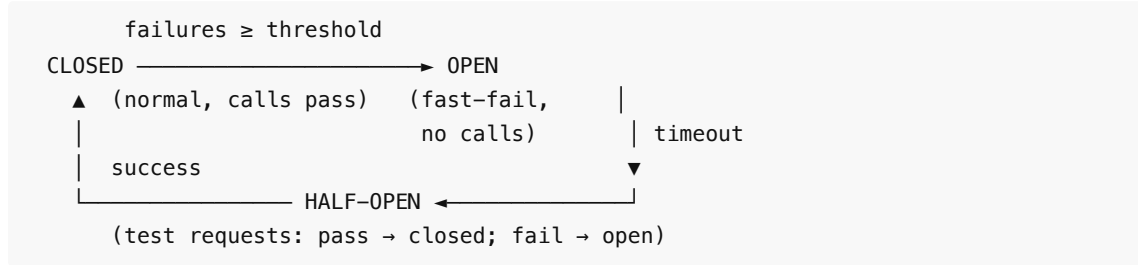
```

```

    })
}

```

Circuit breaker states (diagram)



Resilience patterns — పోలిక

Pattern	సమస్య పరిష్కరిస్తుంది	Go tool
Timeout	slow dependency → cascade	context.WithTimeout
Retry+backoff	transient failures	manual / cenkalti/backoff
Circuit breaker	repeated failures → waste + hammer	sony/gobreaker
Bulkhead	one feature overload → total	separate pools/semaphores
Rate limit	overload / abuse	x/time/rate

Trade-offs

- **Retry amplification:** careless retries load పెంచి outage worsen చేస్తాయి (retry storm). Backoff + jitter + budget + **idempotent only**. Non-idempotent retry → duplicate side-effects.
- **Timeout tuning:** too short → false failures; too long → resources held. Downstream p99 బట్టి. Timeouts stack అవ్వాలి (caller timeout > downstream timeout).
- **Circuit breaker threshold:** too sensitive → unnecessary opens; too lax → cascade delayed. Tune with metrics.
- **Rate limit placement:** per-instance (simple కానీ total = instances × limit) vs distributed (Redis - accurate కానీ network). (Topic 19).

Key Points

- **Timeout every downstream call** (context) - single most important. Timeout లేకపోతే cascade.
- Retry: **exponential backoff + jitter + idempotent only + budget**. Retry storm జాగ్రత్త.
- Circuit breaker: repeated failures → open (fast-fail) → half-open (test) → closed. Downstream recover time.
- Bulkhead: resource isolation (ఒకటి పడితే మిగతావి బతకాలి).
- Rate limit: token bucket (x/time/rate), 429 return.
- **Combine:** timeout + retry + circuit breaker together (breaker retries ని కూడా cap చేస్తుంది).

Interview దృష్టి

"Downstream payment service slow అయితే మొత్తం service down అవుతోంది - fix?" → "Cascading failure - timeout లేకపోవడం. ప్రతి call కి context timeout (2s), circuit breaker (repeated fail → fast-fail, downstream recover time), bulkhead (payment కి separate pool - ఇతర features బతుకుతాయి), idempotent retries

backoff+jitter తో. Fallback (cached/degraded response). ఇవి కలిపి cascade అవుతాయి." Timeout + breaker + bulkhead combo = resilience seniority.

19. Load shedding & graceful degradation

వివరణ

Overload వచ్చినప్పుడు (traffic spike, downstream slow) - service ప్రతి request serve చేయడానికి ప్రయత్నిస్తే అన్నీ slow అయి, అందరూ timeout అయి, service పూర్తిగా crash (metastable failure). బదులుగా:

- **Load shedding** - capacity దాటినప్పుడు కొన్ని requests ని వెంటనే reject (503/429) → మిగతావి healthy గా serve. "కొందరికి fail చేసి అందరికీ fail కాకుండా." Priority-based (VIP/critical requests keep, others shed).
- **Graceful degradation** - non-essential features ని disable చేసి core function కాపాడు. ఉదా: e-commerce లో recommendations service down అయితే, product + "buy" button పనిచేయాలి (recommendations లేకుండా). Fallback to cached/default.

Go లో: bounded queues (full → reject), concurrency limits, AIMD /adaptive limits, priority classification, feature flags/fallbacks.

Adaptive load shedding - static threshold కాదు, actual latency/queue depth చూసి dynamic గా shed (Netflix concurrency-limits, Google CoDel-style).

Real-life Scenario

Load shedding = hospital triage. Mass casualty లో అందరినీ ఒకేసారి treat చేయలేరు (capacity). Nurse triage - critical (heart attack) ముందు, minor (cut finger) wait/redirect. అందరినీ సగంసగం treat చేసి అందరూ చనిపోవడం కంటే, కొందరిని defer చేసి ముఖ్యులను కాపాడటం. **Graceful degradation = restaurant లో AC పోతే** - AC (recommendations) లేకపోయినా భోజనం (core order/pay) serve చేస్తారు, restaurant మూసేయరు. Non-essential పోయినా essential నడుస్తుంది.

Code — concurrency limiter (load shed) + degradation

```
// ---- Load shedding: in-flight concurrency limit (excess → 503) ----
type Limiter struct {
    sem chan struct{} // buffered channel = N permits (bounded)
}

func NewLimiter(max int) *Limiter {
    return &Limiter{sem: make(chan struct{}, max)}
}

func (l *Limiter) Middleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        select {
        case l.sem <- struct{}{}: // permit దొరికింది → serve
            defer func() { <-l.sem }() // release
            next.ServeHTTP(w, r)
        default:
            // capacity full → వెంటనే shed (queue పెట్టి slow అవ్వడం కాదు)
            w.Header().Set("Retry-After", "1")
            http.Error(w, "overloaded, retry later", http.StatusServiceUnavailable)
        }
    })
}
```

```

// 503
    }
    })
}

// ---- Priority-based shedding: critical requests keep, others shed first ----
func (l *Limiter) MiddlewarePriority(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        priority := r.Header.Get("X-Priority") // "critical" vs "normal"
        select {
        case l.sem <- struct{}{}:
            defer func() { <-l.sem }()
            next.ServeHTTP(w, r)
        default:
            if priority == "critical" {
                // critical → కొంచెం wait చేసి try (shed చేయకు)
                select {
                case l.sem <- struct{}{}:
                    defer func() { <-l.sem }()
                    next.ServeHTTP(w, r)
                case <-time.After(200 * time.Millisecond):
                    http.Error(w, "overloaded", http.StatusServiceUnavailable)
                }
                return
            }
            http.Error(w, "overloaded", http.StatusServiceUnavailable) // normal →
            shed
        }
    })
}

```

```

// ---- Graceful degradation: fallback when non-essential fails ----
func ProductPageDegraded(ctx context.Context, id string) *PageData {
    data := &PageData{}
    // core (essential) - fail అయితే error propagate
    data.Product, _ = fetchProduct(ctx, id)

    // recommendations (non-essential) - fail అయితే empty (page ఇంకా పనిచేస్తుంది)
    recCtx, cancel := context.WithTimeout(ctx, 300*time.Millisecond) // tight budget
    defer cancel()
    if recs, err := fetchRecs(recCtx, id); err == nil {
        data.Recs = recs
    } else {
        data.Recs = cachedDefaultRecs() // fallback (stale/generic) - crash కాదు
    }
    return data // recommendations లేకపోయినా page renders
}

```

Load shedding vs Rate limiting

అంశం	Rate limiting	Load shedding
Trigger	pre-set threshold (req/sec)	actual overload (queue/latency)
Goal	fairness, abuse prevent	self-protection (survive spike)
Decision	request rate	current capacity/health
ఉదా	100 req/sec per user	in-flight > N → 503

Trade-offs

- **Shed early, shed fast:** overload లో request queue చేసి slow serve చేయడం worst (అందరూ timeout, resources wasted). వెంటనే 503 → caller retry (backoff) → healthy requests fast.
- **Priority classification cost:** VIP/critical distinguish చేయడానికి request metadata అవసరం (auth tier, endpoint). Complexity.
- **Degradation UX:** feature missing → user notice అవుతారు కానీ "down" కంటే better. Cached/default fallback stale కావచ్చు.
- **Static vs adaptive limits:** static simple కానీ traffic pattern మారితే stale. Adaptive (latency-based) better కానీ complex.

Key Points

- Overload లో: **shed (reject) కొన్ని → మిగతావి healthy.** అందరినీ slow serve = metastable crash.
- Load shedding: capacity full → **fast 503** (queue కాదు). Priority-aware (critical keep).
- Graceful degradation: non-essential fail → fallback/skip, **core function కాపాడు.**
- Go: bounded semaphore/channel → excess reject. default case (non-blocking) లో fast shed.
- Rate limit (threshold, fairness) ≠ load shed (actual overload, self-protect).

Interview దృష్టి

"Black Friday spike - service crash కాకుండా ఎలా?" → "Load shedding + graceful degradation. In-flight concurrency limit - capacity దాటితే వెంటనే 503 (queue పెట్టి అందరూ timeout కాదు), Retry-After లో. Priority - checkout/payment keep, recommendations shed. Non-essential (recs, related items) fail అయితే cached fallback - core buy flow బతుకుతుంది. Autoscale కూడా కానీ shed = immediate protection." Shed-fast + degrade = production survival instinct.

Part 6 — Observability & Operations

20. Structured logging (slog, zap, zerolog), correlation IDs

వివరణ

Production లో "logs = eyes". కానీ **plain text logs** (`fmt.Println`) scale కి పనికిరావు - search/filter/aggregate కష్టం. **Structured logging** = logs ని key-value (JSON) గా → machine-parseable, queryable (Loki/ELK/Datadog లో `status=500 AND user_id=x` filter).

Go options:

- **slog** - Go 1.21+ standard library structured logger. Built-in, no dependency, JSON/text handlers, context support. కొత్త code కి default choice.
- **zap** (Uber) - అత్యంత fast (zero-allocation), production-proven. High-throughput services.

- **zeroLog** - fast, chained API, JSON-first.

Correlation/Trace ID - ఒక request multiple services గుండా వెళ్తుంది. ప్రతి log లో **same request_id/trace_id** ఉంటే - మొత్తం request journey ని అన్ని services logs లో follow చేయొచ్చు. Middleware లో generate/propagate చేసి, **context** ద్వారా అన్ని logs లో include.

Log levels: DEBUG < INFO < WARN < ERROR. Production లో సాధారణంగా INFO+. Structured fields: timestamp, level, message, request_id, user_id, latency, error.

Real-life Scenario

Structured logs = well-labeled పార్సేబుల్ vs విడిగా పడేసిన కాగితాలు. Plain text logs = నేలంతా చిందిన కాగితాల కుప్ప - "500 error ఎక్కడ?" వెతకడం గంటలు. Structured (JSON) logs = ప్రతి కాగితం మీద label (level, request_id, user) - "status=500 AND user_id=123" అని క్షణంలో filter. **Correlation ID = courier tracking number** - ఒక parcel (request) hub A → hub B → hub C (services) గుండా వెళ్తుంది; tracking number (trace_id) ఉంటే మొత్తం journey ఏ hub దగ్గర ఆగింది అని trace చేయొచ్చు. లేకపోతే ఏ service లో fail అయిందో తెలియదు.

Code — slog + context-aware correlation ID

```
package main

import (
    "context"
    "log/slog"
    "net/http"
    "os"
    "time"

    "github.com/google/uuid"
)

// ---- slog setup: JSON, INFO level, source location ----
func setupLogger(env string) *slog.Logger {
    var handler slog.Handler
    opts := &slog.HandlerOptions{Level: slog.LevelInfo, AddSource: true}
    if env == "prod" {
        handler = slog.NewJSONHandler(os.Stdout, opts) // JSON (machine-parseable)
    } else {
        handler = slog.NewTextHandler(os.Stdout, opts) // human-readable (dev)
    }
    return slog.New(handler)
}

// ---- context లో trace_id carry → logs auto-include ----
type ctxKey string

const traceIDKey ctxKey = "trace_id"

// LoggerFromContext: request logger (trace_id తో pre-loaded)
func LoggerFromContext(ctx context.Context, base *slog.Logger) *slog.Logger {
    if id, ok := ctx.Value(traceIDKey).(string); ok {
```

```

        return base.With("trace_id", id) // ప్రతి log line లో trace_id
    }
    return base
}

// TraceID middleware: generate/propagate + context లో పెట్టు
func TraceID(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        id := r.Header.Get("X-Trace-ID")
        if id == "" {
            id = uuid.NewString()
        }
        w.Header().Set("X-Trace-ID", id)
        ctx := context.WithValue(r.Context(), traceIDKey, id)
        next.ServeHTTP(w, r.WithContext(ctx))
    })
}

func handler(base *slog.Logger) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        log := LoggerFromContext(r.Context(), base) // trace_id automatic
        start := time.Now()

        // structured fields (key-value) – queryable
        log.Info("processing_order",
            "user_id", "u123",
            "order_id", "o456",
            "amount_cents", 4999,
        )
        // downstream కి trace_id propagate (header) → distributed correlation
        // req.Header.Set("X-Trace-ID", ...)

        log.Info("request_complete", "duration_ms",
            time.Since(start).Milliseconds())
        w.Write([]byte("ok"))
    }
}

```

Output (prod JSON):

```

{"time":"2026-07-
15T10:00:00Z","level":"INFO","msg":"processing_order","trace_id":"a1b2","user_id":"u123"}

```

Logging libraries — పోలిక

అంశం	slog (stdlib)	zap (Uber)	zerolog
Dependency	none (stdlib 1.21+)	external	external
Performance	good	best (zero-alloc)	very good

API	clean, standard	verbose (typed fields)	chained/fluent
Context support	✓ native	via helpers	via helpers
Best fit	most new code	ultra high-throughput	JSON-heavy, fluent

Logging best practices

✓ చేయి	✗ చేయకు
Structured (key-value/JSON)	plain string concatenation
stdout/stderr కి (12-factor)	file లోకి (infra rotate చేస్తుంది)
trace_id/request_id ప్రతి log	correlation లేకుండా
INFO+ prod, sampling on hot paths	DEBUG spam prod లో
errors తో context (not just "failed")	secrets/PII log (passwords, cards)
levels సరిగ్గా (ERROR = actionable)	అన్నీ ERROR

Trade-offs

- **slog vs zap:** slog stdlib (no dep, good enough for 95%). zap ఉన్నత throughput లో zero-alloc worth it. కొత్త code slog తో మొదలుపెట్టు.
- **Log volume vs cost:** ఎక్కువ logs = ingestion/storage cost + noise. Sampling (hot paths లో 1%), levels తో balance.
- **PII/secrets:** logs లో passwords/cards/tokens NEVER. Redaction/allowlist.
- **Sync vs async logging:** sync (safe, ordered) vs async (fast కానీ crash లో loss). Most libs buffered.

Key Points

- **Structured logging** (JSON, key-value) - queryable, aggregatable. Plain text కాదు.
- slog (Go 1.21+ stdlib) = default; zap = ultra-high-throughput; zerolog = fluent.
- **Correlation/trace ID** middleware లో generate → context → ప్రతి log + downstream headers. Distributed request follow.
- stdout కి log (12-factor); INFO+ prod; **secrets/PII never**.
- Log levels meaningful (ERROR = actionable). Sampling on hot paths.

Interview దృష్టి

"Distributed లో ఒక slow request ని అన్ని services లో ఎలా trace చేస్తావ్?" → "Correlation/trace ID - edge లో generate, context ద్వారా carry, ప్రతి structured log లో include, downstream calls కి header (X-Trace-ID) గా propagate. Log aggregator (Loki/ELK) లో trace_id filter చేస్తే మొత్తం request journey అన్ని services logs లో. Plus OpenTelemetry tracing spans కి link." Structured + correlation + propagation = ops maturity.

21. Metrics (Prometheus client_golang, RED/USE), Grafana

వివరణ

Logs = individual events; **Metrics = aggregated numbers over time** (request count, latency, error rate, CPU). "Is my service healthy? Getting slower?" కి metrics. **Prometheus** (Go లో రాసింది!) = de-facto metrics

system - services /metrics endpoint expose చేస్తాయి, Prometheus scrape చేస్తుంది (pull model), Grafana dashboards + alerting.

Metric types (Prometheus):

- **Counter** - monotonically increasing (requests_total, errors_total). Rate చూస్తాం (rate()).
- **Gauge** - up/down value (in-flight requests, memory, queue depth, temperature).
- **Histogram** - value distribution in buckets (request latency) → percentiles (p50, p99).
- **Summary** - client-side percentiles (histogram preferred సాధారణంగా).

Frameworks to decide WHAT to measure:

- **RED** (services/requests): **R**ate (req/sec), **E**rrors (failed/sec), **D**uration (latency distribution). Request-driven services కి.
- **USE** (resources): **U**tilization, **S**aturation, **E**rrors. CPU/memory/disk/queue కి.
- **Four Golden Signals** (Google SRE): latency, traffic, errors, saturation.

Real-life Scenario

Metrics = car dashboard. Logs = ప్రతి పని జరిగినప్పుడు diary రాయడం (వివరంగా కానీ overwhelming).
Metrics = speedometer, fuel gauge, temperature - ఒక్క చూపులో car health. Speed పెరుగుతోందా (traffic), engine temp ఎక్కువైందా (saturation), warning light (errors)? **RED = ప్రతి రహదారి service కి 3 gauges** - ఎన్ని cars (rate), ఎన్ని accidents (errors), ఎంత సేపు trip (duration). **Prometheus scrape = ప్రతి కొన్ని సెకన్లకి dashboard photo తీయడం** - trend చూడొచ్చు (గంటన్నర క్రితం నుంచి latency పెరుగుతోంది).

Code — Prometheus RED metrics middleware

```
package metrics

import (
    "net/http"
    "strconv"
    "time"

    "github.com/prometheus/client_golang/prometheus"
    "github.com/prometheus/client_golang/prometheus/promauto"
    "github.com/prometheus/client_golang/prometheus/promhttp"
)

var (
    // COUNTER: total requests (rate → traffic; by-status → error rate)
    requestsTotal = promauto.NewCounterVec(prometheus.CounterOpts{
        Name: "http_requests_total",
        Help: "Total HTTP requests",
    }, []string{"method", "path", "status"}) // labels (dimensions)

    // HISTOGRAM: latency distribution → percentiles (p50/p99)
    requestDuration = promauto.NewHistogramVec(prometheus.HistogramOpts{
        Name:    "http_request_duration_seconds",
        Help:    "Request latency",
        Buckets: prometheus.DefBuckets, // .005, .01, ... 10s
    }, []string{"method", "path"})
```

```

// GAUGE: current in-flight requests (saturation)
inFlight = promauto.NewGauge(prometheus.GaugeOpts{
    Name: "http_requests_in_flight",
    Help: "In-flight requests",
})
)

// RED middleware: Rate, Errors, Duration అన్నీ capture
func Middleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        inFlight.Inc() // gauge up
        defer inFlight.Dec() // gauge down (request end)

        start := time.Now()
        rec := &statusRecorder{ResponseWriter: w, status: 200}
        next.ServeHTTP(rec, r)

        path := routePattern(r) // * raw path కాదు (/users/{id}) - high cardinality
        avoid
        requestsTotal.WithLabelValues(r.Method, path,
            strconv.Itoa(rec.status)).Inc() // counter (rate +
errors)
        requestDuration.WithLabelValues(r.Method, path).
            Observe(time.Since(start).Seconds()) // histogram (duration)
    })
}

// /metrics endpoint (Prometheus scrape చేస్తుంది)
func Handler() http.Handler { return promhttp.Handler() }

```

PromQL queries (Grafana లో):

```

# Rate: req/sec (RED-R)
rate(http_requests_total[5m])

# Error rate %: (RED-E)
rate(http_requests_total{status=~"5.."}[5m]) / rate(http_requests_total[5m])

# p99 latency: (RED-D)
histogram_quantile(0.99, rate(http_request_duration_seconds_bucket[5m]))

```

Metric types — పోలిక

Type	ఏమి	ఉదా	Query
Counter	↑ only	requests_total, errors_total	rate()
Gauge	↑↓	in_flight, memory, queue_depth	direct value
Histogram	distribution (buckets)	latency	histogram_quantile()
Summary	client percentiles	(histogram preferred)	direct

RED vs USE

Framework	దేనికీ	Signals
RED	services/requests	Rate, Errors, Duration
USE	resources (CPU, disk, queue)	Utilization, Saturation, Errors
Golden Signals	overall	latency, traffic, errors, saturation

Trade-offs

- **Cardinality explosion:** high-cardinality labels (user_id, raw path with IDs, timestamps) → Prometheus millions time-series → OOM. **Route pattern వాడు (/users/{id}), raw path కాదు.** IDs/emails label లో పెట్టకు.
- **Histogram buckets:** default buckets latency కి సరిపోకపోవచ్చు - service p99 బట్టి custom buckets. చాలా buckets = cost.
- **Pull vs push:** Prometheus pull (scrape) - service discovery easy. Short-lived jobs కి pushgateway.
- **Metrics vs logs:** metrics aggregate (cheap, trends); logs detail (expensive, debugging). రెండూ కావాలి.

Key Points

- Metrics = aggregated numbers (health/trends); Prometheus (pull) + Grafana (viz/alert).
- Types: counter (rate), gauge (up/down), histogram (percentiles). Latency = histogram.
- **RED** (Rate, Errors, Duration) services కి; **USE** resources కి; Golden Signals overall.
- **Cardinality జాగ్రత్త** - route pattern label, raw IDs కాదు (time-series explosion).
- /metrics endpoint + middleware. p99 latency = histogram_quantile .

Interview దృష్టి

"Service ని ఎలా monitor చేస్తావ్?" → "RED metrics - Prometheus client_golang middleware: request rate (counter), error rate (status label), latency (histogram → p50/p99). Gauge for in-flight (saturation). Grafana dashboards + alerts (error rate > 1%, p99 > SLO). Cardinality control - route pattern labels, raw IDs కాదు. Plus USE for resources, distributed tracing for deep dives." RED/USE + cardinality awareness = observability seniority.

22. Distributed tracing (OpenTelemetry), context propagation

వివరణ

Microservices లో ఒక్క user request 10+ services గుండా వెళ్తుంది. "ఎందుకు slow?" - ఏ service? DB? network? Metrics aggregate చెప్తాయి కానీ **single request path** చూపవు. **Distributed tracing** = ఒక request యొక్క end-to-end journey ని **spans** గా capture - ఏ service ఎంత సేపు తీసుకుంది, ఎక్కడ error, ఏ DB call slow.

- **Trace** - ఒక request మొత్తం journey (unique trace_id).
- **Span** - ఒక unit of work (ఒక service call, ఒక DB query). Parent-child hierarchy. Start/end time, attributes, status.
- **Context propagation** - trace_id + span_id ని service-to-service (HTTP headers traceparent , gRPC metadata) carry చేయడం → spans link అవుతాయి.

OpenTelemetry (OTel) = vendor-neutral standard (traces + metrics + logs). Go SDK:

go.opentelemetry.io/otel . Instrument once → any backend (Jaeger, Tempo, Datadog, Honeycomb). **Go context.Context = trace propagation vehicle** - span context ని carry చేస్తుంది.

Real-life Scenario

Distributed trace = courier journey map with timestamps. Parcel (request) Hyderabad hub → Bengaluru hub → local delivery. Tracking (trace) చూస్తే: Hyderabad లో 2 గంటలు (fast), Bengaluru hub లో 2 రోజులు ఆగింది (bottleneck!), delivery 1 గంట. ఏ hub (service) delay చేసిందో exact గా తెలుస్తుంది. Metrics = "సగటు delivery 3 రోజులు" (aggregate) కానీ ఈ parcel ఎక్కడ ఆగిందో trace చెప్తుంది. **Span = ఒక్కో hub stamp** (entry/exit time); **context propagation = tracking number** ప్రతి hub కీ carry అవ్వడం - లేకపోతే hubs విడివిడి, journey link కాదు.

Code — OpenTelemetry tracing + propagation

```
package tracing

import (
    "context"
    "net/http"

    "go.opentelemetry.io/otel"
    "go.opentelemetry.io/otel/attribute"
    "go.opentelemetry.io/otel/trace"
    "go.opentelemetry.io/contrib/instrumentation/net/http/otelhttp"
)

var tracer = otel.Tracer("order-service")

// handler: span create చేసి, downstream కి propagate
func GetOrder(ctx context.Context, orderID string) (*Order, error) {
    // span start (parent = incoming request span, context నుంచి)
    ctx, span := tracer.Start(ctx, "GetOrder")
    defer span.End()
    span.SetAttributes(attribute.String("order.id", orderID)) // searchable
    attribute

    // child span: DB query (nested - hierarchy)
    order, err := fetchFromDB(ctx, orderID)
    if err != nil {
        span.RecordError(err) // error span లో record
        span.SetStatus(codes.Error, "db failed")
        return nil, err
    }

    // downstream service call - ctx carry చేస్తే trace_id auto-propagate
    _ = callPaymentService(ctx, order) // otelhttp client → header లో traceparent
    return order, nil
}

func fetchFromDB(ctx context.Context, id string) (*Order, error) {
    _, span := tracer.Start(ctx, "db.query.orders") // child span
    defer span.End()
    span.SetAttributes(attribute.String("db.statement", "SELECT ... WHERE id=$1"))
    // ...actual query...
```

```

    return &Order{ID: id}, nil
}

// ---- context propagation: HTTP server auto-instrument (otelhttp) ----
func setupServer(mux http.Handler) http.Handler {
    // incoming traceparent header → span context extract; outgoing → inject
    return otelhttp.NewHandler(mux, "http.server")
}

// client: trace context & downstream & inject
var httpClient = http.Client{
    Transport: otelhttp.NewTransport(http.DefaultTransport), // header auto-inject
}

```

Trace hierarchy (Jaeger view):

```

Trace: user-checkout (total 850ms)
├─ http.server GET /checkout [850ms]
│ └─ GetOrder [120ms]
│   └─ db.query.orders [95ms] ← nested
│ └─ callPaymentService [600ms] ← BOTTLENECK
│   └─ http POST /charge [590ms]
│   └─ callShipping [110ms]

```

Tracing vs Metrics vs Logs (three pillars)

Pillar	ఏమి	ఎప్పుడు	Cost
Metrics	aggregated numbers	health, trends, alerts	తక్కువ
Logs	discrete events	debugging, audit	మధ్యం-ఎక్కువ
Traces	request journey (spans)	latency breakdown, dependencies	మధ్యం (sampled)

మూడూ కావాలి: metrics "something wrong" చెప్తాయి → traces "ఎక్కడ" → logs "ఎందుకు".

Trade-offs

- **Tracing overhead + volume:** every request trace చేస్తే huge data. **Sampling** (1-10% లేదా tail-based - slow/error traces keep). Head sampling (start లో decide) vs tail (end - errors keep, expensive).
- **Instrumentation effort:** manual spans work; auto-instrumentation (otelhttp, otel DB drivers) covers common. Custom spans for business logic.
- **Context is everything:** ctx propagate చేయకపోతే spans link కావు (orphan spans). ప్రతి function కీ ctx pass చేయాలి (Go idiom - first param).
- **Vendor lock-in avoid:** OTel standard → backend swap easy (Jaeger → Datadog).

Key Points

- Distributed tracing = single request journey (spans), latency breakdown across services.
- OpenTelemetry = vendor-neutral standard. Instrument once, any backend. Go SDK.
- **context. Context = trace propagation vehicle.** ctx carry చేయకపోతే spans orphan.
- Trace (whole) → spans (units, parent-child). Attributes searchable. Errors recorded.
- **Sampling** (tail-based, errors/slow keep) - cost control. Three pillars కలిపి (metrics→traces→logs).

Interview దృష్టి

"10 services గుండా request slow - ఏ service culprit ఎలా తెలుసుకుంటావ్?" → "Distributed tracing (OpenTelemetry). ప్రతి service span emit చేస్తుంది, context propagation (traceparent header / ctx) తో link. Jaeger/Tempo లో trace చూస్తే span durations - ఏ service/DB call bottleneck (ఉదా payment 600ms). Metrics 'slow' అని alert చేస్తాయి, trace 'ఎక్కడ' చూపిస్తుంది, logs (same trace_id) 'ఎందుకు'. Tail sampling తో slow/error traces keep." Three pillars + context propagation = observability depth.

23. Profiling in production (pprof), runtime metrics

వివరణ

Service slow / memory పెరుగుతోంది / CPU 100% - ఎందుకు? ఏ code? Metrics/traces "ఎక్కడ" చెప్తాయి, **profiling "ఏ function/line"** చెప్తుంది. Go లో profiling **first-class** (runtime/pprof , net/http/pprof) - production లో live profile తీయగలవు (Go biggest ops advantage).

pprof profile types:

- **CPU profile** - ఏ functions CPU ఎక్కువ (hot paths). Sampling-based.
- **Heap profile** - memory allocations (leaks, ఎక్కడ allocate). In-use + alloc.
- **Goroutine profile** - అన్ని goroutines stacks (goroutine leaks - వేలు stuck ఎక్కడ).
- **Block/Mutex profile** - contention (goroutines lock/channel కోసం wait).

net/http/pprof - import చేస్తే /debug/pprof/ endpoints expose అవుతాయి → live production process ని profile (go tool pprof). **flame graphs** తో visualize.

Runtime metrics - runtime.ReadMemStats , runtime/metrics - GC pauses, heap size, goroutine count, GC frequency. Prometheus కి export (Go collector).

Continuous profiling - Pyroscope/Parca/Cloud Profiler - always-on low-overhead profiling (historical - "నిన్న 3pm ఎందుకు slow?").

Real-life Scenario

pprof = body health checkup with X-ray. Doctor (metrics) "నీకు జ్వరం, ఏదో problem" చెప్తాడు. X-ray/scan (pprof) exact organ (function) చూపిస్తుంది - CPU profile = "ఏ కండరం అలసిపోతోంది" (hot function), heap profile = "ఎక్కడ కొప్పు పేరుకుంటోంది" (memory leak), goroutine profile = "ఎన్ని processes stuck" (leaked goroutines). **Continuous profiling = CCTV** - "నిన్న రాత్రి ఏం జరిగింది" చూడొచ్చు (live లో pprof ఇప్పటి snapshot మాత్రమే). Go లో ఈ X-ray machine **built-in + running system మీద** పనిచేస్తుంది (production లో patient ని చంపకుండా).

Code — production pprof + runtime metrics

```
package main

import (
    "expvar"
    "net/http"
    _ "net/http/pprof" // * blank import → /debug/pprof/* handlers register
    "runtime"
    "time"
)
```

```

"github.com/prometheus/client_golang/prometheus/promauto"
"github.com/prometheus/client_golang/prometheus"
)

func main() {
    // pprof ని SEPARATE admin port లో (public కి expose చేయకు - security!)
    go func() {
        // /debug/pprof/profile (CPU), /heap, /goroutine, /block, /mutex
        http.ListenAndServe("localhost:6060", nil)
    }()

    // runtime metrics → Prometheus (Go collector default వస్తుంది; custom కూడా)
    goroutines := promauto.NewGaugeFunc(prometheus.GaugeOpts{
        Name: "app_goroutines",
        Help: "Current goroutine count (leak detect)",
    }, func() float64 { return float64(runtime.NumGoroutine()) })
    _ = goroutines

    // heap size gauge (memory trend)
    promauto.NewGaugeFunc(prometheus.GaugeOpts{
        Name: "app_heap_alloc_bytes",
    }, func() float64 {
        var m runtime.MemStats
        runtime.ReadMemStats(&m) // heap, GC stats
        return float64(m.HeapAlloc)
    })

    // ...main server on :8080...
    select {}
}

// expvar: custom app metrics (uptime, business counters)
var startTime = time.Now()

func init() {
    expvar.Publish("uptime_seconds", expvar.Func(func() any {
        return time.Since(startTime).Seconds()
    }))
}

```

pprof commands (production debugging):

```

# CPU profile (30s live sample) → flame graph
go tool pprof http://localhost:6060/debug/pprof/profile?seconds=30

# heap (memory - leak hunt)
go tool pprof http://localhost:6060/debug/pprof/heap

# goroutine dump (goroutine leak - count పెరుగుతోందా?)
go tool pprof http://localhost:6060/debug/pprof/goroutine

```

```
# interactive: top functions, list <func>, web (flame graph)
(pprof) top10
(pprof) list processOrder    # ఏ line hot
(pprof) web                  # visual flame graph
```

Profile types — ఎప్పుడు ఏది

Profile	Symptom	చూపిస్తుంది
CPU	CPU high, slow	hot functions (సమయం ఎక్కడ)
Heap	memory పెరుగుతోంది / OOM	allocations (ఎక్కడ, ఎంత)
Goroutine	goroutine count పెరుగుతోంది	stuck goroutines (leak location)
Block	latency, contention	channel/sync waits
Mutex	lock contention	mutex hotspots

Trade-offs

- **pprof overhead:** CPU profiling ~expensive కొంచెం (sampling - low, safe in prod). Block/mutex profiling higher overhead (enable temporarily). Heap/goroutine cheap.
- **Security:** /debug/pprof sensitive (stack traces, memory) - **public కి expose చేయకు**. Separate admin port (localhost:6060) + auth. Public port లో register అయితే risk.
- **Snapshot vs continuous:** pprof = ఇప్పటి snapshot ("live crisis" debug). Continuous profiling (Pyroscope/Parca) = historical (past incident). Both.
- **Goroutine leak detection:** goroutine count metric alert (steady పెరుగుదల = leak). pprof goroutine dump తో exact location.

Key Points

- Go profiling first-class (net/http/pprof) - **production live process profile** (Go's ops superpower).
- CPU (hot funcs), heap (allocations/leaks), goroutine (leak detect), block/mutex (contention).
- pprof endpoint **separate admin port + auth** (public కి expose చేయకు - security).
- Runtime metrics (goroutine count, heap, GC pauses) → Prometheus. Goroutine count పెరుగుదల = leak alert.
- Snapshot (pprof live) + continuous profiling (Pyroscope/Parca, historical).

Interview దృష్టి

"Production Go service memory పెరుగుతూ OOM అవుతోంది - ఎలా debug?" → "Live heap profile - `go tool pprof /debug/pprof/heap` (Go లో production-safe). `top / list` తో ఏ function/line allocate చేస్తుందో exact. Goroutine profile - leaked goroutines (context cancel మిస్ అయిందేమో). Runtime metrics - heap trend, GC frequency, goroutine count. Continuous profiling ఉంటే historical. pprof admin port లో, public కాదు." Live pprof = Go-specific senior ops skill.

Part 7 — Deploy & Scale

24. Containerizing Go — multi-stage, distroless/scratch, CGO_ENABLED

వివరణ

Go's killer deploy feature: **single static binary** → **tiny container**. కానీ naive Dockerfile (FROM golang లో build + run) = 1 GB image (whole Go toolchain + source). **Multi-stage build** తో final image లో **binary మాత్రమే** → 5-20 MB.

- **Multi-stage build** - stage 1 (builder): full Go image లో compile. Stage 2 (runtime): minimal base లో binary copy మాత్రమే. Toolchain/source final image లో ఉండదు.
- **Base image choice:**
 - **scratch** - empty (0 bytes). Static binary మాత్రమే. Smallest కానీ shell/certs/tools లేవు (debug కష్టం).
 - **distroless** (Google) - minimal (no shell/package manager) కానీ CA certs, tzdata, /etc/passwd ఉన్నాయి. Secure + practical. **Recommended.**
 - **alpine** - tiny (~5MB) కానీ musl libc (CGO issues). Shell ఉంది (debug).
- **CGO_ENABLED=0** - pure Go, no C dependencies → truly static binary (scratch/distroless లో పనిచేస్తుంది). CGO on (default) → glibc dynamic linking → scratch లో crash. Cross-compile easy కి CGO off.

Real-life Scenario

Multi-stage = వంటగదిలో వండి, plate మాత్రమే బయటికి తీసుకెళ్ళడం. Naive Dockerfile = వంటగది మొత్తం (stove, ingredients, utensils = Go toolchain, source) dining table కి మోసుకెళ్ళడం - భారీ, అనవసరం. Multi-stage = వంటగదిలో (builder stage) వండి, పూర్తయిన భోజనం (binary) ఒక్క plate (final image) లో serve. **scratch = భాళీ plate** (భోజనం మాత్రమే, fork/spoon లేవు = shell లేదు). **distroless = plate + fork + napkin** (essentials: certs, tzdata) కానీ వంటగది కాదు.

Code — production multi-stage Dockerfile

```
# ---- Stage 1: builder (full Go toolchain) ----
FROM golang:1.22-alpine AS builder

WORKDIR /src

# deps ముందు (layer cache: go.mod మారకపోతే deps re-download కాదు)
COPY go.mod go.sum ./
RUN go mod download

COPY . .

# static binary: CGO_ENABLED=0 (no C deps), stripped (-s -w = smaller)
RUN CGO_ENABLED=0 GOOS=linux go build \
    -ldflags="-s -w -X main.version=$(git rev-parse --short HEAD)" \
    -trimpath \
    -o /app ./cmd/api

# ---- Stage 2: runtime (minimal - binary మాత్రమే) ----
FROM gcr.io/distroless/static-debian12:nonroot
```

```
# distroless: no shell, no package manager (attack surface చిన్నది),
# కానీ CA certs + tzdata + /etc/passwd ఉన్నాయి (HTTPS, timezones పనిచేస్తాయి)

COPY --from=builder /app /app      # binary ఒక్కటే copy (source/toolchain కాదు)

USER nonroot:nonroot              # non-root (security - root గా run చేయకు)
EXPOSE 8080
ENTRYPOINT ["/app"]
# final image ~15-25 MB (binary + minimal base)
```

```
# ---- Alternative: scratch (absolute smallest) ----
FROM scratch
# certs manually copy చేయాలి (HTTPS కి - scratch లో ఏమీ లేదు)
COPY --from=builder /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/
COPY --from=builder /app /app
ENTRYPOINT ["/app"]
# ~5-15 MB కానీ shell లేదు (kubectl exec debug కష్టం)
```

Base image — పోలిక

Base	Size	Shell	Certs	CGO	Debug	Security
scratch	~0 (binary only)	✗	✗ (manual)	✗ needs static	కష్టం	best (nothing)
distroless/static	~2 MB	✗	✓	✗ static	కష్టం	very good
alpine	~5 MB	✓	✓	musl issues	easy	good
debian-slim	~30 MB	✓	✓	✓	easy	ok

Trade-offs

- **scratch vs distroless:** scratch smallest కానీ certs/tzdata manual + zero debug tools. distroless = 99% cases best (certs, non-root, minimal, secure). Alpine debug easy కానీ musl/CGO glibc issues.
- **CGO on/off:** off = static, tiny image, easy cross-compile. On = C libs (sqlite, some crypto) కానీ dynamic linking → scratch break, larger. Default off unless C dep అవసరం.
- **Image size vs debuggability:** tiny (scratch) = no `kubectl exec sh`. Ephemeral debug containers (k8s) లేదా distroless:debug variant.
- **Layer caching:** go.mod copy + download ముందు (deps rarely change) → build fast. Source మారితే deps re-download అవ్వకూడదు.

Key Points

- **Multi-stage build:** builder (compile) → minimal runtime (binary only). 1GB → 15MB.
- `CGO_ENABLED=0` → static binary → scratch/distroless. Cross-compile easy.
- **distroless recommended** (certs, non-root, minimal, no shell = secure). scratch smallest but bare.
- `-ldflags="-s -w"` (strip debug) + `-trimpath` → smaller + reproducible. Version inject via ldflags.
- Non-root user (security). Layer cache: go.mod deps ముందు.

Interview దృష్టి

"Go service ని ఎలా containerize చేస్తావ్?" → "Multi-stage: builder stage లో CGO_ENABLED=0 static binary, final stage distroless/static (certs+non-root, no shell = minimal attack surface). ~15MB image. Idflags -s -w strip + version inject. Layer cache go.mod ముందు. scratch smallest కానీ debug/certs కష్టం - distroless practical." Multi-stage + CGO + distroless = deploy maturity.

25. Running Go in Kubernetes — GOMAXPROCS, GOMEMLIMIT, probes

వివరణ

Kubernetes లో Go run చేయడంలో **critical gotcha**: Go runtime ని configure చేసేవి (**GOMAXPROCS**, **GOMEMLIMIT**) **host machine** ని చూస్తాయి, container **cgroup limits** ని కాదు (default). Misconfig → throttling, OOM kills.

- **GOMAXPROCS** - simultaneous goroutine-executing OS threads (= parallelism). Default = **host CPU count** (`runtime.NumCPU()`). కానీ container కి `cpu: 2` limit ఉంటే, 64-core node మీద GOMAXPROCS=64 → Go 64 threads run చేస్తుంది కానీ cgroup 2 cores కి throttle → **excessive context-switching, latency, CPU throttling**. Fix: **uber-go/automaxprocs** - cgroup CPU limit చూసి GOMAXPROCS set చేస్తుంది.
- **GOMEMLIMIT** (Go 1.19+) - soft memory limit. GC ఈ limit కి approach అయితే aggressive గా collect చేస్తుంది → OOM kill avoid. Default off. Container memory limit కి ~90% set చేయి (headroom). లేకపోతే Go heap పెరిగి cgroup limit దాటి **OOMKilled** (GC too late).
- **Resource requests/limits** - CPU/memory requests (scheduling) + limits (cap). Go కి memory limit ముఖ్యం (OOM), CPU limit throttling.
- **Probes** - liveness (restart), readiness (traffic), startup (slow start) - Topic 3.

Real-life Scenario

GOMAXPROCS mismatch = 64 మంది cooks కి 2 stoves. Container ని 64-core node మీద run చేస్తే, Go (default) "64 cooks (threads) పెట్టు" అనుకుంటుంది - కానీ cgroup limit "నీకు 2 stoves (cores) మాత్రమే" (`cpu: 2`). 64 cooks 2 stoves కోసం తోసుకుంటారు (context switching), పని నెమ్మది + throttled. `automaxprocs` = "నీకు 2 stoves ఉన్నాయి కాబట్టి 2 cooks చాలు" అని సరిచేస్తుంది. **GOMEMLIMIT = fridge నిండకముందే వంట తగ్గించడం** - fridge (memory limit) నిండిపోతే landlord ఇంటినుంచి గెంటీస్తాడు (OOMKilled). GOMEMLIMIT = "fridge 90% నిండింది, ఇక cleanup (GC) చెయ్" అని ముందే warn.

Code — automaxprocs + GOMEMLIMIT + k8s manifest

```
package main

import (
    "runtime/debug"

    _ "go.uber.org/automaxprocs" // ★ blank import → GOMAXPROCS = cgroup CPU limit
    // (init లో container CPU quota చదివి runtime.GOMAXPROCS set చేస్తుంది)
)

func main() {
    // GOMEMLIMIT: env var GOMEMLIMIT=450MiB పెట్టొచ్చు, లేదా code లో:
    debug.SetMemoryLimit(450 << 20) // 450 MiB soft limit (container limit 512Mi కి ~90%)
    // GC ఈ limit దగ్గర aggressive → OOMKilled avoid
}
```

```
// ...server setup...
}
```

```
# k8s deployment - Go-tuned
apiVersion: apps/v1
kind: Deployment
spec:
  template:
    spec:
      containers:
      - name: api
        image: myservice:v1
        resources:
          requests:
            # scheduler ఇంత reserve చేస్తుంది
            cpu: "500m"
            memory: "256Mi"
          limits:
            # cap (దాటితే throttle/OOM)
            cpu: "1000m"      # automaxprocs → GOMAXPROCS=1
            memory: "512Mi"   # GOMEMLIMIT ~450Mi (headroom)
        env:
        - name: GOMEMLIMIT
          value: "450MiB"      # container memory కి ~88% (safe)
        # ---- probes (Topic 3) ----
        livenessProbe:
          # fail → restart (deadlock detect మాత్రమే)
          httpGet: { path: /healthz, port: 8080 }
          periodSeconds: 10
          failureThreshold: 3
        readinessProbe:
          # fail → traffic ఆపు (deps check)
          httpGet: { path: /readyz, port: 8080 }
          periodSeconds: 5
        startupProbe:
          # slow start కి (liveness/readiness delay)
          httpGet: { path: /healthz, port: 8080 }
          failureThreshold: 30  # 30 × 2s = 60s startup grace
          periodSeconds: 2
        terminationGracePeriodSeconds: 30  # > app shutdown timeout (graceful)
```

Go-in-K8s gotchas

సమస్య	కారణం	పరిష్కారం
CPU throttling, high latency	GOMAXPROCS = host cores (not cgroup)	automaxprocs
OOMKilled	Go heap పెరిగి memory limit దాటింది	GOMEMLIMIT ~90% limit
Cascade restarts	liveness లో DB check	liveness thin (deadlock only)
Traffic mid-deploy dropped	no graceful shutdown	Topic 3 (readiness+drain)

Slow startup restart	startup > liveness threshold	startupProbe
----------------------	------------------------------	--------------

CPU/Memory limits — Go behavior

Limit	Go default behavior	Fix
CPU limit	GOMAXPROCS = host CPUs (throttle)	automaxprocs (cgroup-aware)
Memory limit	GC unaware → OOMKilled	GOMEMLIMIT (soft, GC aggressive)

Trade-offs

- **GOMEMLIMIT headroom:** too close to limit (99%) → GC thrashing (constant GC, CPU burn). Too low (50%) → OOM risk లేదు కానీ memory underused. ~90% sweet spot. It's a **soft** limit (GC works harder, but can still OOM under burst).
- **CPU limit vs no limit:** CPU limit → predictable but throttling. Some (Tim Bray, others) argue no CPU limit + automaxprocs = better latency (no throttle) కానీ noisy-neighbor risk. Requests always set.
- **GOMAXPROCS=1 (cpu:1):** fine for I/O-bound (goroutines block on network, 1 thread చాలు) కానీ CPU-bound work serialize అవుతుంది. Sizing workload బట్టి.

Key Points

- **automaxprocs** తప్పనిసరి k8s లో - GOMAXPROCS ని cgroup CPU limit కి align (throttling/latency fix).
- **GOMEMLIMIT** ~90% of memory limit - GC aggressive → OOMKilled avoid (Go 1.19+).
- Resource requests (schedule) + limits (cap). CPU limit → throttle; memory limit → OOM.
- Probes: liveness (thin, restart), readiness (deps, traffic), startup (slow boot).
terminationGracePeriodSeconds > shutdown timeout.
- Default Go runtime **container-unaware** - ఈ రెండూ (automaxprocs + GOMEMLIMIT) prod essentials.

Interview దృష్టి

"Go service k8s లో latency spikes + occasional OOMKills - fix?" → "రెండు classic Go-in-k8s issues. Latency: GOMAXPROCS host cores చూస్తోంది (cgroup CPU limit కాదు) → threads > allowed cores → CPU throttling + context switching. automaxprocs (cgroup-aware) fix. OOM: Go GC memory limit unaware → heap పెరిగి OOMKilled. GOMEMLIMIT ~90% of limit → GC aggressive. రెండూ set చేస్తా." automaxprocs + GOMEMLIMIT = Go-in-cloud senior signal.

26. Scaling Go services — statelessness, autoscaling, GC/allocation tuning

వివరణ

Scaling = ఎక్కువ load handle చేయడం. రెండు దిశలు: **vertical** (bigger machine - limit ఉంది) vs **horizontal** (more instances - preferred, cloud-native).

- **Statelessness** = horizontal scaling కి precondition. Instance లో request-specific state (session, in-memory data) పెట్టకు → ఏ instance అయినా ఏ request అయినా serve చేయొచ్చు → freely add/remove instances behind LB. State (sessions, cache) → external (Redis, DB). Go services naturally stateless గా design చేయి.

- **Horizontal scaling** = replicas పెంచడం (behind LB). Go stateless service → `replicas: N`. Linear scale (ideally).
- **Autoscaling (HPA)** - Kubernetes Horizontal Pod Autoscaler - CPU/memory/custom metrics (req/sec, queue depth) బట్టి replicas auto adjust. Go fast startup → autoscale responsive.
- **GC/allocation tuning** - Go GC low-pause కానీ high-allocation code GC pressure పెంచుతుంది (CPU, latency). Reduce allocations: `sync.Pool` (reuse objects), preallocate slices (`make([]T, 0, n)`), avoid unnecessary pointers/interface boxing, GOGC tuning. `pprof` heap profile → allocation hotspots.

Real-life Scenario

Stateless = ఏ counter అయినా ఏ customer ని అయినా serve చేయగలడు. Bank లో ప్రతి counter customer file తన drawer లో పెట్టుకుంటే (stateful) - ఆ customer మళ్ళీ అదే counter కే రావాలి (sticky), ఆ counter busy అయితే wait. బదులుగా files central record room (Redis/DB) లో ఉంటే (stateless), ఏ counter అయినా serve చేయగలడు - busy అయితే కొత్త counter తెరవొచ్చు (add instance), ఖాళీ అయితే మూసేయొచ్చు (autoscale down). **GC tuning** = చెత్త ఎక్కువ ఉత్పత్తి చేయకపోవడమే మంచిది - ప్రతి request కి కొత్త objects (allocations) చేస్తే, cleaner (GC) పడేపడే రావాలి (CPU). Reusable plates (sync.Pool) వాడితే చెత్త తక్కువ, cleaner తక్కువ అంతరాయం.

Code — stateless + sync.Pool + preallocation

```
// ---- Stateless: state external (Redis session), instance లో కాదు ----
type Server struct {
    sessions *redis.Client // session Redis లో (in-memory map కాదు!)
    db        *sql.DB
}

func (s *Server) handler(w http.ResponseWriter, r *http.Request) {
    sessionID := r.Header.Get("X-Session")
    // state external నుంచి → ఏ instance అయినా serve చేయగలడు (horizontal scale)
    data, _ := s.sessions.Get(r.Context(), "session:"+sessionID).Result()
    _ = data
    // ⚠ చేయకు: var sessions = map[string]Session{} (instance-local → sticky అవసరం)
}

// ---- sync.Pool: object reuse → allocation/GC pressure తగ్గింపు ----
var bufferPool = sync.Pool{
    New: func() any { return new(bytes.Buffer) }, // pool ఖాళీ అయితే కొత్తది
}

func processRequest(data []byte) []byte {
    buf := bufferPool.Get().(*bytes.Buffer) // reuse (కొత్త alloc కాదు)
    defer func() {
        buf.Reset() // clear
        bufferPool.Put(buf) // తిరిగి pool కి (next request reuse)
    }()
    buf.Write(data)
    // ...process...
    return append([]byte(nil), buf.Bytes()...)
}
```

```
// ---- Preallocation: capacity ముందే తెలిస్తే allocate ఒక్కసారే ----
func collectIDs(users []User) []string {
    ids := make([]string, 0, len(users)) // cap ముందే → append re-alloc/copy లేదు
    for _, u := range users {
        ids = append(ids, u.ID)
    }
    return ids
    // make([]string, 0) అయితే append పడేపడే grow (multiple allocs + copies)
}
```

```
# HPA: CPU/custom metric బట్టి autoscale
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  scaleTargetRef: { kind: Deployment, name: api }
  minReplicas: 3
  maxReplicas: 50
  metrics:
  - type: Resource
    resource:
      name: cpu
      target: { type: Utilization, averageUtilization: 70 } # CPU 70% → scale up
# custom: req/sec, queue depth (KEDA తో Kafka lag కూడా)
```

Scaling approaches

అంశం	Vertical	Horizontal
ఎలా	bigger machine	more instances
Limit	hardware ceiling	~unlimited
Availability	SPOF	redundant
Precondition	-	statelessness
Go fit	-	✅ (fast start, small)

GC/allocation tuning levers

Lever	ఏమి చేస్తుంది	ఎప్పుడు
sync.Pool	object reuse (buffers, temp)	high-alloc hot paths
Preallocate (make(_, 0, n))	slice re-alloc/copy తగ్గింపు	known size
gogc (default 100)	GC frequency (↑ = తక్కువ GC, ఎక్కువ mem)	memory-vs-CPU tradeoff
GOMEMLIMIT	soft mem cap	OOM avoid

Reduce pointers/interface boxing	escape-to-heap తగ్గింపు	allocation profile
----------------------------------	-------------------------	--------------------

Trade-offs

- **Statelessness cost:** state external (Redis) → network hop + Redis dependency. కానీ horizontal scale + availability worth it. Sticky sessions avoid (instance చస్టి session పోతుంది).
- **sync.Pool caveats:** GC pool ని clear చేయొచ్చు (guaranteed reuse కాదు), objects reset చేయాలి (stale data leak), premature optimization avoid (profile ముందు). High-alloc paths లోనే.
- **GOGC tuning:** GOGC↑ (200) → GC అరుదు (CPU తక్కువ) కానీ memory ఎక్కువ. GOGC↓ → memory తక్కువ కానీ CPU ఎక్కువ. GOMEMLIMIT + default GOGC సాధారణంగా చాలు.
- **Premature optimization:** allocation tuning ముందు **profile** (pprof). Guessing waste. చాలా services కి stateless + horizontal + caching చాలు; GC micro-tuning అరుదుగా అవసరం.

Key Points

- **Statelessness = horizontal scaling precondition.** State external (Redis/DB), instance-local కాదు. Sticky sessions avoid.
- Horizontal (replicas + LB) > vertical. Go fast start + small → autoscale-friendly (HPA).
- GC/alloc tuning: sync.Pool (reuse), preallocate slices, reduce pointers. **Profile ముందు** (pprof heap) - guess కాదు.
- HPA: CPU/memory/custom (req/sec, queue). GOGC + GOMEMLIMIT memory-CPU balance.
- చాలా apps కి stateless + caching + horizontal చాలు; micro GC tuning last resort.

Interview దృష్టి

"Go service ని 10x traffic కి ఎలా scale?" → "Stateless (state Redis/DB - horizontal enable) → replicas పెంచి behind LB → HPA (CPU/req-per-sec autoscale). Go fast startup కాబట్టి autoscale responsive. Caching (read-heavy). Bottleneck DB అయితే read replicas/pooler. Allocation hotspots ఉంటే pprof heap profile → sync.Pool/preallocate, కానీ ముందు profile - premature tuning కాదు." Stateless + horizontal + profile-first = scaling maturity.

27. Security basics — TLS, secrets, validation, JWT/OAuth

వివరణ

Security = HLD లో cross-cutting concern. Go లో core practices:

- **TLS** - transport encryption. Public: TLS termination at LB/ingress. Internal service-to-service: **mTLS** (mutual TLS - both sides verify) - service mesh (Istio/Linkerd) or manual. Go crypto/tls first-class. Always HTTPS, TLS 1.2+.
- **Secrets** - DB passwords, API keys, JWT signing keys. **Code/git లో NEVER.** Env vars (k8s Secret), Vault, cloud secret manager. Rotate periodically.
- **Input validation** - never trust client input. Validate/sanitize (type, range, format). SQL injection → **parameterized queries** (\$1 , never string concat). XSS → output encoding. Struct validation (go-playground/validator).
- **AuthN vs AuthZ** - Authentication (ఎవరు నువ్వు - login) vs Authorization (ఏం చేయగలవు - permissions).
- **JWT** - stateless token (signed claims). Login → server signed JWT ఇస్తుంది → client ప్రతి request లో పంపుతుంది → server verify (signature + expiry). Stateless (DB lookup అవసరం లేదు) కానీ revocation కష్టం (expiry దాకా valid).

- **OAuth2/OIDC** - delegated auth (Google/GitHub login). Go: golang.org/x/oauth2 , [coreos/go-oidc](https://coreos.com/go-oidc) .

Real-life Scenario

JWT = concert wristband. Entry దగ్గర ID verify చేసి (login) tamper-proof wristband (signed token) ఇస్తారు. లోపల ప్రతి stall దగ్గర wristband చూపిస్తే చాలు (stateless - మళ్ళీ ID check అవసరం లేదు), signature (hologram) fake కాదని verify చేస్తారు. కానీ wristband ఒకసారి ఇస్తే event అయ్యేదాకా valid - మధ్యలో ban చేయడం కష్టం (revocation problem). **Secrets in code = ఇంటి తాళంవెలి doormat కింద** - convenient కానీ ఎవరైనా (git history, leaked repo) దొరకపుచ్చుకోవచ్చు. Vault/secret manager = bank locker. **Parameterized query = form లో blanks నింపడం** - user input ని SQL "command" గా కాకుండా "data" గా treat (injection ఆగుతుంది).

Code — JWT auth + validation + TLS

```
// ---- JWT auth middleware ----
import "github.com/golang-jwt/jwt/v5"

var jwtSecret = []byte(os.Getenv("JWT_SECRET")) // * env నుంచి (code లో కాదు!)

func JWTAuth(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        auth := r.Header.Get("Authorization")
        tokenStr, ok := strings.CutPrefix(auth, "Bearer ")
        if !ok {
            http.Error(w, "missing token", http.StatusUnauthorized)
            return
        }
        token, err := jwt.Parse(tokenStr, func(t *jwt.Token) (any, error) {
            // * algorithm verify (alg=none attack అవు)
            if _, ok := t.Method.(*jwt.SigningMethodHMAC); !ok {
                return nil, fmt.Errorf("unexpected signing method")
            }
            return jwtSecret, nil
        })
        if err != nil || !token.Valid { // signature + expiry verify
            http.Error(w, "invalid token", http.StatusUnauthorized)
            return
        }
        claims := token.Claims.(jwt.MapClaims)
        ctx := context.WithValue(r.Context(), "user_id", claims["sub"])
        next.ServeHTTP(w, r.WithContext(ctx))
    })
}

// ---- Input validation (struct tags) ----
import "github.com/go-playground/validator/v10"

type CreateUserReq struct {
    Email string `json:"email" validate:"required,email"`
    Age   int    `json:"age" validate:"gte=0,lte=130"`
    Name  string `json:"name" validate:"required,min=1,max=100"`
}
```

```

}

var validate = validator.New()

func createUser(w http.ResponseWriter, r *http.Request) {
    var req CreateUserReq
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        http.Error(w, "bad json", http.StatusBadRequest)
        return
    }
    if err := validate.Struct(req); err != nil { // validate ముందు (trust కాదు)
        http.Error(w, "validation: "+err.Error(), http.StatusBadRequest)
        return
    }
    // * parameterized query (SQL injection safe - string concat ఎప్పుడూ కాదు)
    db.ExecContext(r.Context(),
        `INSERT INTO users (email, name) VALUES ($1, $2)`, req.Email, req.Name)
}

// ---- TLS server (min 1.2) ----
srv := &http.Server{
    Addr:    ":8443",
    Handler: handler,
    TLSConfig: &tls.Config{
        MinVersion: tls.VersionTLS12, // 1.2+ మాత్రమే (పాతవి insecure)
    },
}
srv.ListenAndServeTLS("cert.pem", "key.pem")

```

JWT vs Session

అంశం	JWT (stateless)	Session (stateful)
Storage	client (token)	server (session store)
Scale	easy (no lookup)	session store (Redis)
Revocation	కష్టం (expiry దాకా valid)	easy (delete session)
Size	large (claims)	small (id)
Best fit	microservices, APIs	traditional web, revocation ముఖ్యం

Security checklist (Go)

Area	✓	✗
Transport	TLS 1.2+, mTLS internal	plain HTTP
Secrets	env/Vault, rotate	hardcode, git
SQL	parameterized (\$1)	string concat

Input	validate + sanitize	trust client
JWT	verify alg + expiry	alg=none, no expiry
Passwords	bcrypt/argon2	plain/MD5
Errors	generic to client	stack traces/internal leak
Deps	govulncheck, scan	ignore CVEs

Trade-offs

- **JWT revocation:** stateless = fast కానీ compromised token expiry దాకా valid. Mitigations: short expiry + refresh tokens, revocation list (stateful-ish), token versioning. High-security → sessions.
- **mTLS cost:** strong internal auth కానీ cert management (rotation, distribution) - service mesh offloads.
- **TLS termination:** at LB (simple, internal plain) vs end-to-end (mTLS everywhere, secure కానీ complex). Zero-trust → mTLS.
- **Validation everywhere:** every boundary (API, queue consumer) validate - defense in depth. Trust nothing.

Key Points

- TLS 1.2+ always; internal service-to-service mTLS (service mesh). Go `crypto/tls` first-class.
- **Secrets:** `env/Vault/secret-manager`, **NEVER code/git**. Rotate.
- **Parameterized queries** (SQL injection). Input validation ప్రతి boundary (`validator`). Trust nothing.
- JWT = stateless (scale easy, revocation hard); session = stateful (revocation easy). Verify alg + expiry (alg=none attack).
- Passwords: bcrypt/argon2. `govulncheck` for dependency CVEs. Generic errors to client.

Interview దృష్టి

"Microservices auth ఎలా?" → "Edge (API gateway) లో OAuth2/OIDC login → JWT issue. Downstream services JWT verify (signature + expiry + alg check). Internal service-to-service mTLS (service mesh - identity + encryption). Secrets Vault/k8s Secret (code కాదు). JWT revocation కష్టం కాబట్టి short expiry + refresh tokens. Input validate ప్రతి boundary, parameterized queries." JWT + mTLS + secrets + revocation awareness = security seniority.

Part 8 — Reference

28. HLD-in-Go interview framework + checklist + Memory Tips + Common Mistakes + numbers cheat sheet

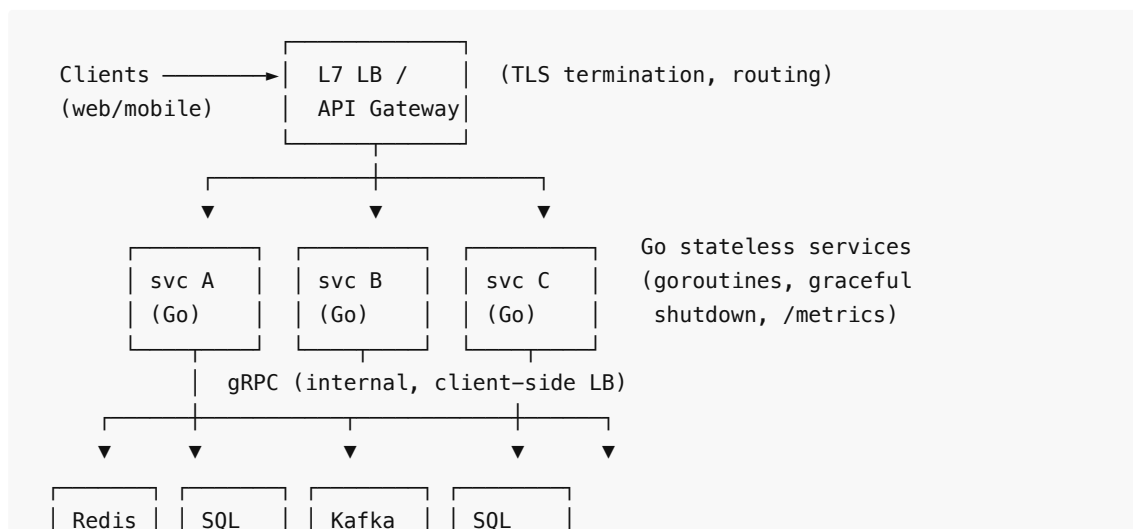
వివరణ

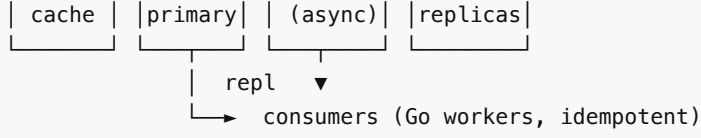
ఇది capstone reference - ఒక HLD-in-Go interview / design task ని approach చేయడానికి structured framework, plus quick-recall tables. HLD interview లో generic system design (Requirements → Estimation → API → Data → Diagram → Deep Dive → Wrap up) చేసేటప్పుడు, **Go-specific implementation depth** ని layer చేస్తే seniority చూపిస్తుంది.

HLD-in-Go interview framework (step-by-step)

1. REQUIREMENTS (5 min)
 - Functional (ఏ features) + Non-functional (scale, latency, consistency).
 - Read-heavy vs write-heavy? Realtime? Consistency (money=strong)?
 - Scope narrow చేయి (assumptions చెప్పు).
2. ESTIMATION (5 min)
 - DAU → QPS (peak). Storage/year. Bandwidth.
 - Go concurrency: "ఒక్క instance ~లక్షల goroutines/connections".
3. API DESIGN (5 min)
 - REST (public/browser) vs gRPC (internal service-to-service).
 - Versioning (/v1/), pagination (cursor), idempotency (keys).
4. DATA MODEL (5 min)
 - SQL (relations/ACID) vs NoSQL (scale). Schema.
 - Go: database/sql pool, sqlc for type-safety.
5. HIGH-LEVEL DIAGRAM
 - client → LB (L7) → Go stateless services → cache (Redis) → DB (+ replicas).
 - Async: → Kafka → consumers.
6. DEEP DIVE (15 min) – ఇక్కడ Go depth చూపించు
 - Concurrency: worker pools, errgroup fan-out, bounded (semaphore).
 - Resilience: context timeouts, circuit breaker, retries+backoff.
 - Caching: cache-aside + singleflight (stampede).
 - Scaling: stateless + horizontal + HPA. Sharding (last resort).
 - Ops: graceful shutdown, GOMAXPROCS/GOMEMLIMIT, pprof, RED metrics.
7. WRAP UP (5 min)
 - Bottlenecks, SPOFs, failure modes, monitoring (RED + tracing).
 - Trade-offs (ఎందుకు ఇది, ఇంకోటి కాదు).

High-level architecture (reference diagram)





Observability: Prometheus (RED) + Grafana + OTel traces (Jaeger/Tempo) + slog

Go HLD decision cheat sheet

అవసరం	Go choice
Public API	REST (chi/stdlib), TLS at edge
Internal service-to-service	gRPC (protobuf, HTTP/2, client-side LB)
Realtime push	SSE (notifications) / WebSocket (chat) + Redis backplane
Read-heavy	Redis cache-aside + singleflight + read replicas
Write-heavy	Kafka async + worker consumers (idempotent)
Parallel sub-calls	errgroup fan-out/fan-in
Bounded work	worker pool / semaphore
Slow dependency	context timeout + circuit breaker + bulkhead
Overload	load shed (503) + graceful degradation
DB access	database/sql pool (tuned) + sqlc
Config	env → typed struct (12-factor), Vault secrets
Deploy	multi-stage distroless (CGO=0), ~15MB
K8s	automaxprocs + GOMEMLIMIT + probes + graceful shutdown
Scale	stateless + horizontal + HPA
Observability	slog + Prometheus (RED) + OTel tracing + pprof

Latency numbers (గుర్తుంచుకో)

Operation	Time (approx)
L1 cache reference	~1 ns
Mutex lock/unlock	~25 ns
Main memory (RAM) reference	~100 ns
Channel send/recv (Go)	~50-100 ns
Goroutine creation	~1-3 μs
SSD random read	~100 μs (0.1 ms)

Redis GET (same DC)	~0.5-1 ms
Network round-trip (same DC)	~0.5 ms
DB query (indexed, same DC)	~1-10 ms
HDD seek	~10 ms
Network round-trip (cross-region)	~50-150 ms

పాఠం: RAM » SSD » network(DC) » network(cross-region). Cache (RAM/Redis) వేగం; cross-region calls ఖరీదు. Go goroutine/channel nanoseconds - cheap (concurrency free గా వాడొచ్చు కానీ bounded).

Capacity numbers

అంశం	విలువ
సెకన్లు/రోజు	86,400 (10 ⁵)
1M req/day	~12 QPS
1B req/day	~11,600 QPS
Goroutines/instance	లక్షలు (~2KB each)
Concurrent connections/Go server	~100K-500K (I/O-bound)
Go image size (distroless)	~15-25 MB
Go startup	milliseconds

Availability downtime

Availability	Downtime/year
99%	3.65 రోజులు
99.9%	8.75 గంటలు
99.99%	52 నిమిషాలు
99.999%	5 నిమిషాలు

Memory Tips (గుర్తుంచుకునే analogy)

Concept	గుర్తుంచుకోవడానికి
Go binary	రెడీమేడ్ tiffin box (deploy = ఒక్క file)
Goroutine	చవకైన worker (లక్షలు, 2KB)
GOMAXPROCS	ఎన్ని stoves (cores) - cooks కాదు
GOMEMLIMIT	fridge నిండకముందే GC
automaxprocs	"నీకు 2 stoves మాత్రమే" సరిచేయడం

Graceful shutdown	shop మూసేటప్పుడు లోపలివాళ్ళను drain
Liveness vs Readiness	సజీవమా (restart) vs traffic-ready (ఆపు)
Worker pool	fixed ticket counters (bounded)
errgroup fan-out	వంట parallel stoves మీద (latency ↓)
Circuit breaker	fuse (fail fast, cascade ఆపు)
Bulkhead	పడవ watertight compartments
Singleflight	ఒక్కరు market కి పోయి అందరికీ తేవడం
Cache-aside	fridge ముందు చూడు, లేకపోతే market
Idempotency key	ATM withdrawal token (double charge కాదు)
Fencing token	toilet key number (highest valid)
Saga	order = pay+stock+ship, fail → compensate
Distributed lock TTL	key మర్చిపోతే spare key timeout
gRPC	typed telegram (compact, internal)
Cursor pagination	bookmark (page number కాదు)
Distributed tracing	courier journey map (ఏ hub slow)
pprof	body X-ray (ఏ function hot)
Load shedding	hospital triage (కొందరిని defer)
Stateless	ఏ counter అయినా ఏ customer అయినా
Sharding	library branches (last resort)
Raft quorum	committee majority vote

Common Mistakes (వీటిని చేయకు)

❌ తప్పు	✅ సరైనది
bare <code>http.ListenAndServe</code>	<code>http.Server{}</code> explicit timeouts
unbounded <code>go f()</code> per request	worker pool / semaphore (bounded)
context pass చేయకపోవడం	ప్రతి call కి ctx (timeout, trace, cancel)
downstream call timeout లేకుండా	<code>context.WithTimeout</code> (cascade ఆపు)
goroutine leak (ctx watch లేదు)	<code><-ctx.Done()</code> cleanup
<code>rows.Close()</code> / <code>tx.Rollback()</code> మర్చిపోవడం	defer (connection leak)
ప్రతి request కి <code>sql.Open</code>	ఒక్కసారి pool, share

GOMAXPROCS host-aware (k8s)	automaxprocs (cgroup)
GOMEMLIMIT లేకుండా (OOMKilled)	~90% memory limit
liveness లో DB check	thin liveness (cascade restart avoid)
secrets in code/git	env/Vault
string-concat SQL	parameterized (\$1)
non-idempotent retry	idempotent + dedup keys
cache stampede (no protection)	singleflight
stateful instance (in-mem session)	external state (Redis) - stateless
microservices/sharding by default	monolith/single-DB first (YAGNI)
high-cardinality metric labels	route pattern, no raw IDs
graceful shutdown లేకుండా deploy	readiness off + drain
GORM blind (N+1, opaque SQL)	sqlc (type-safe) / Preload

ఒక్క వాక్యంలో ముఖ్య సూత్రాలు

- **Go ఎందుకు:** static binary + concurrency + low memory + fast start = cloud-native king.
- **Concurrency:** bounded (pool/semaphore), never unbounded. errgroup for parallel.
- **Context everywhere:** timeout + cancel + trace + values - ప్రతి call లో.
- **Resilience:** timeout + retry(backoff) + circuit breaker + bulkhead + load shed.
- **Caching:** cache-aside + singleflight + TTL (read-heavy).
- **Data:** SQL first, pool tuned, shard last resort.
- **Scale:** stateless + horizontal + HPA + caching.
- **K8s:** automaxprocs + GOMEMLIMIT + probes + graceful shutdown.
- **Deploy:** multi-stage distroless (CGO=0), tiny image.
- **Observe:** slog + RED metrics + OTel traces + pprof.
- **Everything is a trade-off** - free lunch లేదు. ప్రతి decision justify చేయి.

Interview దృష్టి — SDE2 vs SSE (Go lens)

అంశం	SDE2	SSE
Concurrency	goroutines/channels వాడగలడు	bounded + backpressure + leak-aware
Resilience	timeout/retry తెలుసు	cascade/metastable failure reasoning
Ops	deploy చేయగలడు	GOMAXPROCS/GOMEMLIMIT/pprof/shutdown depth
Trade-offs	తెలుసు	ప్రతి Go decision justify (ఎందుకు gRPC, ఎందుకు sqlc)
Scale	horizontal scale	statelessness, sharding trade-offs, DB limits
Failure	happy path	failure modes proactive (SPOF, drain, degrade)

కీలకం: SDE2 "Go లో ఎలా build చేయాలి" చూపిస్తాడు. SSE "ఎందుకు ఈ Go pattern, fail అయితే ఏమవుతుంది, container/k8s లో ఎలా behave చేస్తుంది, ఎలా observe/scale చేస్తా" అని reasoning చూపిస్తాడు. Go-in-production depth (bounded concurrency, context propagation, GOMAXPROCS, graceful shutdown, pprof) = seniority signal.

ముగింపు

గుర్తుంచుకో:

- Go cloud-native king ఎందుకంటే: **static binary + native concurrency + low memory + fast start**. Docker/K8s/etcd/Prometheus అన్నీ Go.
- HLD-in-Go = classic building blocks (LB, cache, shard, CAP) + **Go-specific mastery** (bounded goroutines, context propagation, GOMAXPROCS/GOMEMLIMIT, graceful shutdown, pprof, tiny containers).
- Concurrency = capacity, infinite కాదు** - ఎప్పుడూ bounded (pool/semaphore). Context ప్రతి call లో (timeout/cancel/trace).
- Resilience:** timeout + circuit breaker + bulkhead + load shed. Cascade/metastable failure ని ఆలోచించు.
- Requirements → Estimation → API → Data → Diagram → **Go deep dive** → Trade-offs. ఈ క్రమం మర్చిపోకు.
- SDE2 = correct Go design; **SSE = correct + justified + resilient + observable + operable in production/k8s**.
- "Perfect design" లేదు - justified design ఉంది. Numbers + Go internals తో reasoning = seniority.

ఈ documents కలిస్తే - **Go language → LLD (Go) → HLD (Go) → System Design (Go)** - ఒక complete తెలుగు Go engineering reference:

- G0_Telugu.md - Go language (goroutines, channels, interfaces) — **ముందు చదువు**
- LLD_Go_Telugu.md - Low-Level Design in Go (interfaces, patterns)
- HLD_Go_Telugu.md - High-Level Design in Go (ఈ document)
- SystemDesign_Go_Telugu.md - Case studies (Go implementations)

HLD (High-Level Design) in Go - Complete Telugu Guide (SDE2 & SSE)